

Материалы кафедры ММП факультета ВМК МГУ. Введение в глубокое обучение.

Занятие 16. Рекуррентные нейронные сети

Занятие провел: Дегтев Василий
(@vasco_de_gtev)

Материалы составили: Дегтев Василий
(@vasco_de_gtev), Феокистов Дмитрий
(@trandelik), Алексеев Илья (@voorhs),
Оганов Александр (@welud) на основе
материалов Хисматуллина Владимира
(s02190724@gse.cs.msu.ru)

Москва, Весенний семестр 2026

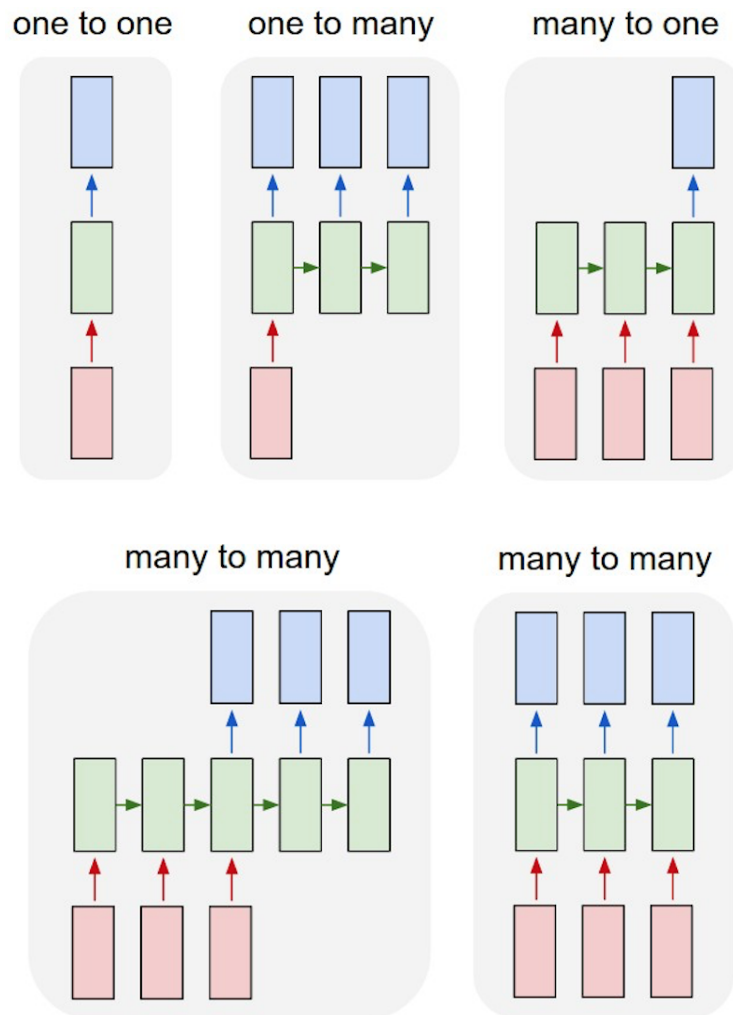
Источники:

- **Simple Recurrent Networks (Elman):** [Elman J. L., 1991](#), примерно 2 тысячи цитирований
- **LSTM:** [Hochreiter S., Schmidhuber J., 1997](#), примерно 146 тысяч цитирований
- **GRU:** [Cho K. et al., 2014](#), примерно 40 тысяч цитирований
- **Foundations of RNN and LSTM**, [Alex Shertinsky, 2023](#), примерно 5 тысяч цитирований
- [Beam search](#)
- [Подсчет градиентов для разных видов RNN](#)
- [Открытый гитхаб курса по DL на coursera](#)

Данный семинар будет посвящен практическому применению RNN в различных задачах по обработке последовательностей. Семинар будет состоять из трех частей: вводная теоретическая часть, практика с различными видами RNN и обучение

посимвольной языковой модели на основе RNN. В начале вспомним основные свойства RNN и какие типы задач они решают, разберем проблемы затухания и взрыва градиентов и как с ними бороться. Далее вспомним архитектуры GRU и LSTM и в чем их преимущество по сравнению с классической RNN Элмана. В практической части напишем цикл обучения с клипингом градиента и логгированием нормы градиента, обучим и сравним RNN Элмана, LSTM, GRU в различных задачах. В последней части вспомним задачу Causal Language Modeling и познакомимся со стратегиями декодирования: greedy, beam search и top-k sampling. В конце обучим собственную посимвольную языковую модель на текстах Шекспира и реализуем жадное декодирование.

Основные модели работы с последовательностями



1. One-to-one: На вход один объект и на выходе тоже (вырожденный случай)
2. One-to-many: На вход подается один объект, на выходе последовательность объектов
3. Many-to-one: На вход подается последовательность объектов, на выходе один объект
4. Many-to-many: На входе и выходе последовательности нефиксированной длины

Вопрос: Приведите примеры задач каждого вида

Ответ:

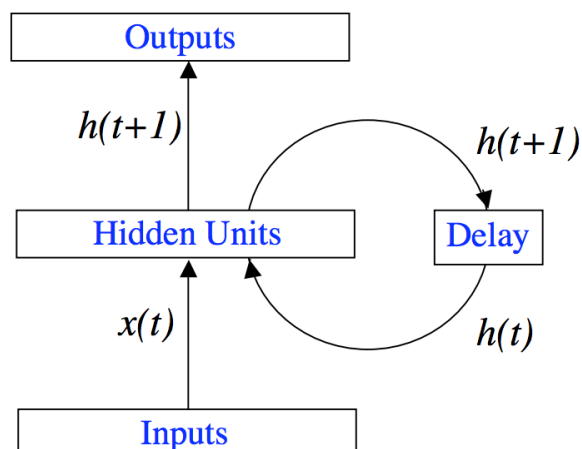
- One-to-one: классификация картинок
- One-to-many: создание подписей к картинкам (image captioning)
- Many-to-one: классификация текстов или видео
- Many-to-many: машинный перевод, суммаризация

Вопрос: Какие основные свойства RNN?

Ответ:

- Parameter sharing. Если в картинках такое хорошо работает, то хотелось бы что-то такое и тут
- Возможность обрабатывать последовательности любой длины
- Обогащение векторных представлений токенов информацией об окружающих токенах
- Нельзя распараллелить на GPU

Рекуррентные нейронные сети



Простая рекуррентная нейронная сеть (сеть Элмана):

$$h_t = \tanh(W_{ih}^T \cdot X_t + W_{hh}^T \cdot h_{t-1} + b_h), \quad (1)$$

$$\hat{y}_t = \text{softmax}(W_{oh}^T \cdot h_t + b_y), \quad (2)$$

где W_{ih} , W_{hh} и W_{oh} – матрицы весов соответственно для входа, скрытых состояний и выхода.

Небольшой трюк, повышающий эффективность реализации модели Элмана

Пусть $n_{neurons}$ – размерность скрытого состояния (количество нейронов в скрытом состоянии).

1. Матрицы весов W_{ih} и W_{hh} можно конкатенировать вертикально и получить W размера $(n_{inputs} + n_{neurons}) \times n_{neurons}$.
2. Обозначим конкатенацию входа X_t и скрытого состояния h_{t-1} размера $m \times (n_{inputs} + n_{neurons})$ как $[X_t, h_{t-1}]$.

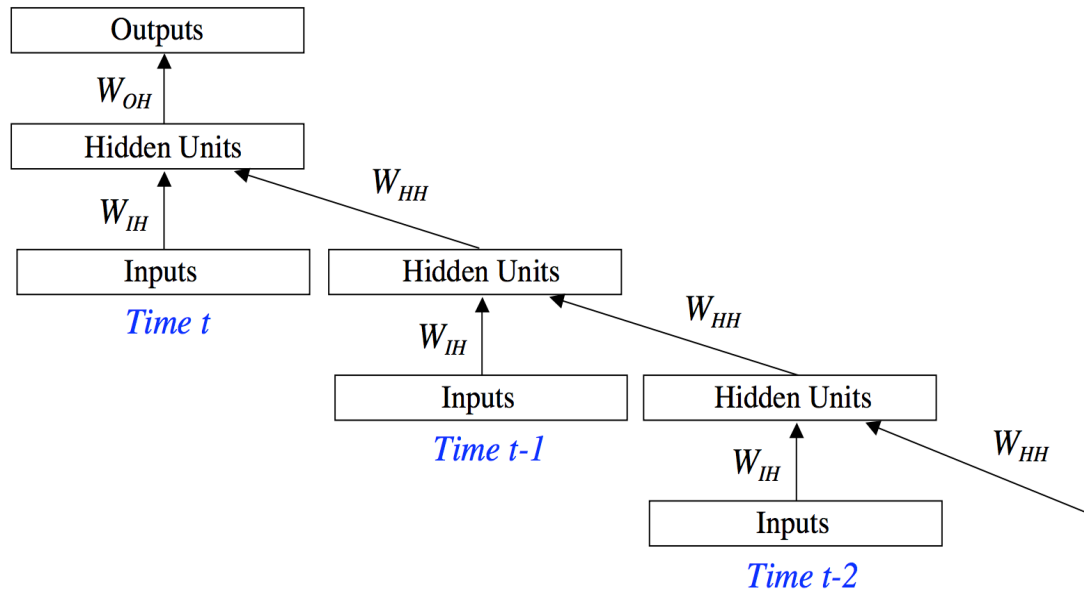
Тогда новое состояние можно вычислить по обновлённой формуле:

$$\begin{aligned} h_t &= \tanh(X_t \cdot W_{ih} + h_{t-1} \cdot W_{hh} + b_h) \\ &= \tanh([X_t, h_{t-1}] \cdot W + b_h) \end{aligned}$$

Резюмируем:

1. X_t размера $m \times n_{inputs}$ матрица с проекциями всех последовательностей в момент t .
2. h_{t-1} размера $m \times n_{neurons}$ матрица с скрытыми состояниями для **каждого** объекта в батче в момент t .
3. W_{ih} размера $n_{inputs} \times n_{neurons}$ матрица весов между входом и текущим скрытым слоем.
4. W_{hh} размера $n_{neurons} \times n_{neurons}$ матрица весов между прошлым и текущим скрытыми слоями.
5. W_{oh} размера $n_{neurons} \times n_{output}$ матрица весов между текущим скрытым состоянием и выходом.
6. b_h вектор размера $n_{neurons}$ смещение для обновления скрытого слоя.
7. b_o вектор размера n_{output} смещение для вычисления выхода модели.
8. y_t размера $m \times n_{output}$ матрица выхода модели в момент t для **каждого** объекта в батче.

Развертывание рекуррентной сети



Источник

Используя введенную ранее рекурсивную формулу опишем состояние модели в момент времени t :

$$h_t = f_h(X_t, f_h(X_{t-1}, f_h(X_{t-2}, \dots (X_2, f_h(X_1, h_0))))$$

$$h_t = \tanh(W_{ih}^T \cdot X_t + W_{hh}^T \cdot \tanh(\dots (\tanh(W_{ih}^T \cdot X_1 + W_{hh}^T \cdot h_1 + b_h) + b_h)))$$

Как бороться с взрывом градиентов?

Помимо использования LSTM и GRU (– т.н. gated архитектуры), полезный трюк это gradient clipping.

Обычный шаг обучения:

1. семплируем батч данных: $B \sim \text{Data}$
2. проход вперед: $\mathcal{L} := \text{NN}_\theta(B)$
3. проход назад: $g := \nabla_\theta \mathcal{L}$
4. шаг оптимизации: $\theta_{\text{new}} := \theta_{\text{old}} - \eta g$

Объект g – это вектор, в который собрали градиенты всех весов сети. Обрезка градиента заключается в модификации градиента g следующим образом:

$$\hat{g} = \begin{cases} g, & \text{если } \|g\|_2 \leq \theta, \\ \theta \frac{g}{\|g\|_2}, & \text{если } \|g\|_2 > \theta, \end{cases}$$

где θ – максимальное значение нормы градиента, которое мы допускаем. Если норма градиента на данном батче оказалась больше положенного, то градиенты всех параметров сети масштабируются таким образом, чтобы вектор g имел норму θ .

Вопрос: Как вы думаете, почему бы просто не применить покомпонентный `torch.clamp` ?

Ответ:

Тогда изменится направление градиента.

Вопрос: Как бороться с затуханием градиентов?

Ответ:

- Оптимизация с помощью методов с инерцией и надежда на то, что мы из-за нее проскочим место, где градиенты тухнут
- Использование функций активации с единичными производными (ReLU, Hardshrink)
- Ортогональная инициализация (попытка удержания единичной нормы за счёт инициализации)
- **Gated models – наверное, самый эффективный, простой и популярный способ. Использование сетей специального вида (LSTM, GRU).**

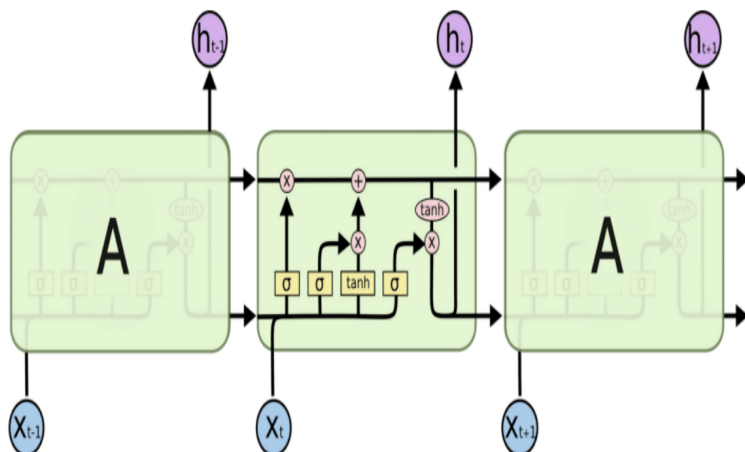
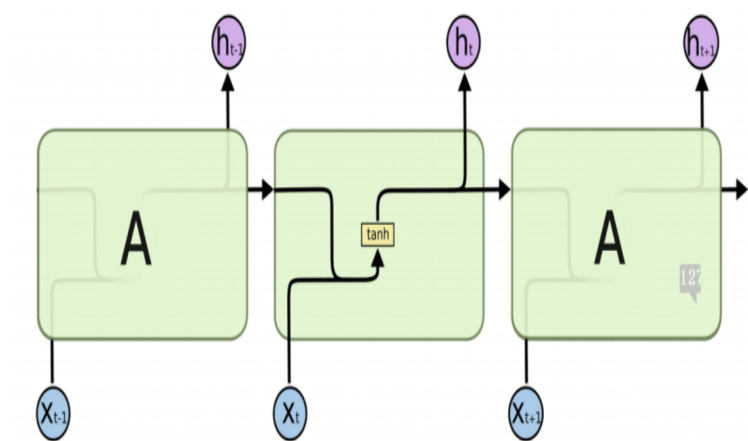
Если в случае взрывов градиентов у нас есть универсальный способ борьбы – клиппинг, то для затухания такого нет, и пункт про архитектуру становится еще важнее

Модель LSTM

Standard RNN



LSTM



Источник

Точные формулы:

$$\text{forget } f_t = \text{Sigmoid}(W_f x_t + U_f h_{t-1} + b_f), \quad (5)$$

$$\text{input } i_t = \text{Sigmoid}(W_i x_t + U_i h_{t-1} + b_i), \quad (6)$$

$$\text{output } o_t = \text{Sigmoid}(W_o x_t + U_o h_{t-1} + b_o), \quad (7)$$

$$\text{candidate } \tilde{C}_t = \text{Tanh}(W_c x_t + U_c h_{t-1} + b_c), \quad (8)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad (9)$$

$$h_t = o_t \odot \text{Tanh}(C_t). \quad (10)$$

Разбор смысловой нагрузки гейтов

1. Forget gate

Регулирует, какие сигналы (компоненты C_t) нужно ослабить (механизм стирания из памяти).

Допустим, на вход модели поступает текст. Она должна следить за семантической структурой, а потому когда слова меняются с единственного числа на множественные,

то модель должна забыть часть прошлого контекста. Представим, что некоторые координаты $1, \dots, m$ из представления C_t отвечают за число и пол субъекта части предложения.

Рассмотрим предложение "Гости съели пироги – постарался наврать Вова". В первой части подлежащее множественного числа, а во второй части единственного.

В тот момент, когда пришло слово 'постарался', мы понимаем, что речь идёт уже о другом субъекте. Поэтому f_t будет примерно равна 0 для координат C отвечающих за пол, число, так как их нужно забыть.

2. Input (update) gate

Данный gate отвечает за то, будет ли внесена информация из Candidate в C_t .

Например, на слове 'пироги' нам не нужно обновлять число субъекта, поэтому $i_t \approx 0$. Но на слове 'постарался' необходимо зафиксировать изменения: $i_t \approx 1$.

3. Output gate

В отличие от остальных gate'ов, данный не влияет на C_t , но меняет h_t . Регулирует, что должно остаться в короткой памяти и пойти на выход. Например, если среди координат $1, \dots, m$ в C_t отвечающих за число текущего субъекта также хранится информация о предыдущих субъектах, то данный gate должен её удалить.

Модель GRU

Точные формулы:

$$\text{reset } r_t = \text{Sigmoid}(W_{ir}x_t + W_{hr}h_{t-1} + b_r), \quad (11)$$

$$\text{update } z_t = \text{Sigmoid}((W_{iz}x_t + W_{hz}h_{t-1} + b_z), \quad (12)$$

$$\text{new } n_t = \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})), \quad (13)$$

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}. \quad (14)$$

В отличие от LSTM, GRU состоит только из трех гейтов и не поддерживает глобальный контекст, всю необходимую информацию мы содержим в скрытом состоянии.

Вопрос: Попробуйте догадаться, что делает каждый из гейтов, исходя из формул и знаний LSTM

Ответ:

- Update Gate (z): Он определяет, в каком виде прошлую информацию надо передать далее. По сути является аналогом комбинации input и output gate в LSTM
- Reset Gate (r): определяет, какую часть прошлых знаний следует забыть. По сути действует аналогично комбинации input gate и forget gate в LSTM

- New Gate (n): часто не выделяют, как отдельный гейт, а объединяют с Update Gate, что следует из названия. Здесь происходит добавление новой информации к старой и определение того, что мы хотим передать далее с учетом этих знаний

GRU очень похожа на LSTM, но использует меньше параметров из-за чего ее обучение дешевле. Эти две архитектуры являются самыми популярными среди RNN

Обработка последовательностей в PyTorch

Теперь посмотрим на практике то, что обсуждалось ранее. А именно поведение градиентов и методов борьбы с их плохим поведением

Данные: даны x , y , z координаты частей рук человека.

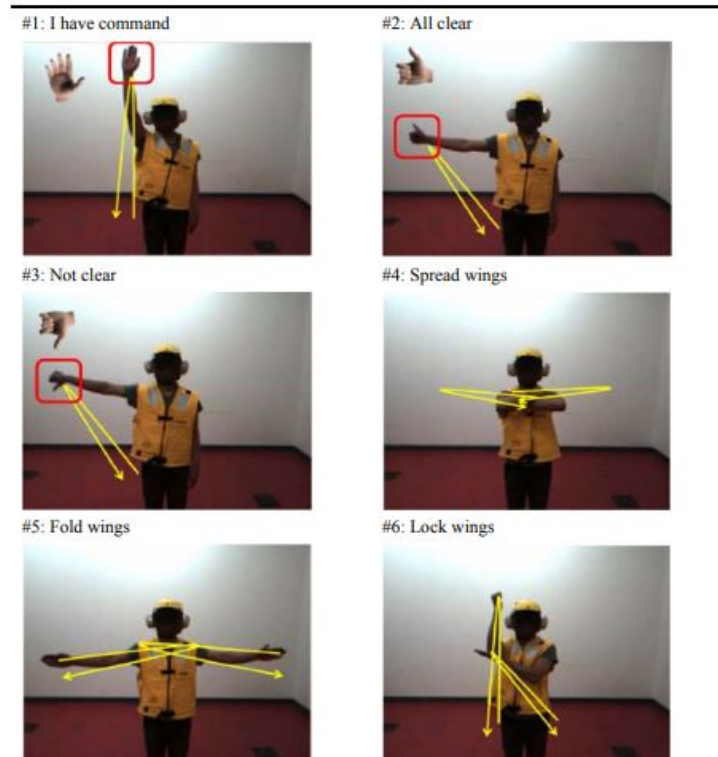
$\mathbb{X} = (x_1, x_1, \dots, x_L)$ – последовательность фиксированной длины из многомерных векторов,

$x = (x_{1,1}, x_{1,2}, x_{1,3}, \dots, x_{8,1}, x_{8,2}, x_{8,3}) \in \mathbb{R}^{24}$ – проекция последовательности в некоторый момент времени.

Задача: Классифицировать жесты, выполняемые человеком (всего 6 классов).

[Датасет NATOPS.](#)

Table 1 The six selected gestures from the NATOPS database; figures have been extracted from (see online version for colours)



Source: Song et al. (2011)

Данная задача – классификация временного ряда.

Тип задачи: 'many to one'

В данной простой задаче длины рядов как раз совпадают для всех объектов и равны 51, значит можно успешно решать свёртками/полносвязными сетями как one to one, так как длина ряда фиксирована. Мы же попробуем решить с помощью RNN.

```
In [ ]: # !pip install sktime -q
```

```
In [2]: from collections import defaultdict
import pandas as pd
import numpy as np
import scipy

import os
from pathlib import Path
import string
import re

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
from torchvision import transforms as T
from torch.optim.lr_scheduler import StepLR

import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns

import sktime
from sktime.datasets import load_from_tsfile_to_dataframe
from torch.utils.data import Dataset, DataLoader

from tqdm.notebook import tqdm
import random
from math import floor

import gc
```

```
In [3]: if not os.path.exists('NATOPS'):
        os.system('wget https://www.timeseriesclassification.com/aeon-toolkit/NATOPS.zip')
        os.system('unzip NATOPS.zip -d NATOPS')
        os.system('rm NATOPS.zip')
```

```
In [3]: %config InlineBackend.figure_formats = ['svg']
```

```
In [4]: def set_global_seed(seed: int) -> None:
        """
        Set global seed for reproducibility.
        """
```

```

torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
random.seed(seed)
np.random.seed(seed)

set_global_seed(42)

```

```

In [5]: def seed_worker(worker_id):
        worker_seed = torch.initial_seed() % 2**32
        np.random.seed(worker_seed)
        random.seed(worker_seed)

        g = torch.Generator()
        g.manual_seed(42)

```

```

Out[5]: <torch._C.Generator at 0x72e1fe402990>

```

Для работы с данными в PyTorch следует писать свой Dataset.

В данном случае исходные данные имеют тип sktime. Используя соответствующую библиотеку их можно отразить в pandas.DataFrame. Оттуда их нужно перевести в numpy или сразу torch.

Особенность обработки – метки могут быть строками, поэтому для перевода их в числа нужно хранить словарь, который нужно 'обучить' на основной части выборки и использовать в тестовой.

```

In [7]: class Sktime_Series_Classification_Dataset(Dataset):
        def __init__(self, data_path, label_dict=None, transforms=None, pad_len=None):
            """
            :data_path: Строка к файлу с данными в формате sktime
            :label_dict: Словарь преобразования исходных меток (строк) в индексы
            :transforms: Возможные аугментации данных
            :pad_len - length of padding to apply when the length is not fixed
            """
            super().__init__()

            # Данные приходят размера N x M x L
            _x, _y = load_from_tsfile_to_dataframe(data_path)

            self.M = _x.shape[1]
            self.series_len = _x[_x.columns[0]].iloc[0].shape[0]

            # Если нужно привести все ряды к одинаковой длине - добиваем до заданной
            if pad_len is not None:
                self.X = [[np.hstack((np.array(_x.iloc[object_ind][column]), np.zeros(
                    pad_len - _x[_x.columns[0]].iloc[object_ind].shape[0]))
                    for column in _x.columns for object_ind in range(_x[_x.columns[0]].shape[0])
                else:
                    self.X = [[np.array(_x.iloc[object_ind][column]) for column in _x.columns]
                    for object_ind in range(_x[_x.columns[0]].shape[0])
                self.X = np.array(self.X)

            # Создаём словарь для обработки классов

```

```

if label_dict is None:
    self.label_dict = {}
    for i, word in enumerate(np.unique(_y)):
        self.label_dict[word] = i
    print('Размерность обучающей выборки {0} x {1} x {2} (Количество слов {0} x {1} x {2})'.format(*self.X.shape))
    print('Всего {0} классов'.format(len(self.label_dict.keys())))
else:
    # В тестовый датасет передадим готовый словарь
    self.label_dict = label_dict

# Кодировать вектор таргетов как (-1,)
self.y = np.zeros((_y.shape[0]), int)
for i, word in enumerate(_y):
    self.y[i] = self.label_dict[word]

self.transforms = transforms

def __getitem__(self, idx):
    """
    :idx: индекс объекта из выборки
    """
    object_ = [self.X[idx].astype(np.float32), self.y[idx]]

    # Если есть преобразования - применяем
    if self.transforms is not None:
        for transform in self.transforms:
            object_ = transform(object_)

    return object_

def __len__(self):
    """
    :return int: number of objects in dataset
    """
    return self.X.shape[0]

```

Для применения трансформаций выборки необходимо реализовать специальный декоратор.

```

In [8]: def basic_series_transform(transform):
    """
    Применяет преобразование transform к временному ряду.
    Метку при этом не трогает.
    """
    def wrapper(object, *args, **kwargs):
        object[0] = transform(object[0], *args, **kwargs)
        return object
    return wrapper

class Jittering(object):
    """
    Добавляет нормальный шум в ряд.
    """
    def __init__(self, sigma):
        self.sigma = sigma

```

```
def __call__(self, object):
    Noise = np.random.normal(loc=0, scale=self.sigma, size=object.shape)
    return object + Noise
```

Важные замечания:

1. Нами выбрана и зафиксирована концепция 'Batch first'. Т.е. классы реализованы таким образом, что в сеть передаваться будет 'объект' размера $N \times L \times M$. В домашнем задании будет другой вариант, в котором 'объект' будет размера $L \times N \times M$. При обработке временных последовательностей свёртками также популярно представление $N \times M \times L$.
2. Если ваша модель рекуррентной сети работает (не выдаёт ошибку), но не обучается (качество скачет около минимума) – вполне возможно, что вы передали данные неправильного вида. Например в сеть, которая принимает батчи размера $N \times L \times M$ передали данные вида $L \times N \times M$.
3. В данном, упрощённом случае длина L одинакова для всех батчей. Более общий случай будет разобран на следующих занятиях.

```
In [9]: # Считываем всю информацию в заданные классы

name = 'NATOPS'
train_path = name + "/" + name + "_TRAIN.ts"
test_path = name + "/" + name + "_TEST.ts"

transforms = [basic_series_transform(Jittering(0.1)), basic_series_transform]

train_dataset = Sktime_Series_Classification_Dataset(train_path, transforms=transforms)
test_dataset = Sktime_Series_Classification_Dataset(test_path, train_dataset=train_dataset)

test_loader = DataLoader(test_dataset, batch_size=10, worker_init_fn=seed_worker)
train_loader = DataLoader(train_dataset, batch_size=10, drop_last=True, shuffle=True)
```

Размерность обучающей выборки 180 x 24 x 51 (Количество объектов x размерность x длина)

Всего 6 классов

Цикл обучения

```
In [6]: # Просто функции обучения модели
# Единственное отличие от стандартной парадигмы -
# при итерации по батчу не приходят пары (data, target),
# а словарь с соответствующими полями
@torch.no_grad()
def evaluate(model, device, data_loader):
    """
    Возвращает качество и лосс модели на выборке
    """
    model.eval()
    loss = 0
```

```

correct = 0
length = 0

for batch_idx, (data, target) in enumerate(data_loader):
    data, target = data.to(device), target.to(device)
    output = model(data)

    # Обновляем качество
    loss += F.cross_entropy(output, target, reduction="sum").item()
    pred = output.argmax(dim=1)
    correct += (pred == target).sum().item()
    length += target.numel()

return loss / length, correct / length

def train(model, device, train_loader, test_loader, n_epoch, optimizer, scheduler,
          max_norm=None, track_gradient=False):
    """
    Обучение и оценивание качества на test и train одновременно

    max_norm - параметр gradient clipping. Если None, то метод не используется
    track_gradient - Если true, то оцениваем градиент на каждом батче.
    """
    train_history, test_history = {'loss':[], 'acc':[]}, {'loss':[], 'acc':[]}

    if track_gradient:
        grad_history = np.zeros((n_epoch, len(list(model.parameters()))))

    for epoch in tqdm(range(1, n_epoch + 1)):
        n_objects = 0
        model.train()
        for batch_idx, (data, target) in enumerate(train_loader):
            data, target = data.to(device), target.to(device)

            # Делаем шаг по батчу
            optimizer.zero_grad()
            output = model(data)
            loss = F.cross_entropy(output, target, reduction="mean")
            loss.backward()

            # Gradient clipping
            if max_norm is not None:
                torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=max_norm)

            # Если нужно - запоминаем норму градиента (отдельно по каждому параметру)
            if track_gradient:
                n_objects += data.shape[0]
                for i, p in enumerate(model.parameters()):
                    param_norm = p.grad.data.detach().norm(2)
                    grad_history[epoch-1][i] += param_norm.item() ** 2

            optimizer.step()
            scheduler.step()

        # Добавляем информацию в наш логгер
        loss, acc = evaluate(model, device, train_loader)

```

```

train_history['loss'].append(loss), train_history['acc'].append(acc)
loss, acc = evaluate(model, device, test_loader)
test_history['loss'].append(loss), test_history['acc'].append(acc)

if track_gradient:
    for i, p in enumerate(model.parameters()):
        grad_history[epoch-1][i] = (grad_history[epoch-1][i] / n_obj

if track_gradient:
    return train_history, test_history, grad_history
return train_history, test_history

```

Обратите внимание, что `torch.nn.utils.clip_grad_norm_` возвращает норму градиентов до нормирования, мы же сейчас хотим отслеживать норму градиентов после нормирования.

```

In [7]: def delete_model(model: nn.Module):
        """
        source: https://discuss.pytorch.org/t/how-can-we-release-gpu-memory-cache
        """
        model.cpu()
        del model
        gc.collect()
        torch.cuda.empty_cache()

```

Построение рекуррентных моделей в PyTorch

```

In [12]: class My_LSTM_Layer(nn.Module):
        """
        Класс - работа RNN с произвольной LSTM-ячейкой
        Единственное требование от этой ячейки - вход и выход как у LSTM

        Формат входа как в Batch first LSTM в pytorch - (N, L, M)
        """
        def __init__(self, input_size, hidden_size, rnn_cell, batch_first=False):
            super().__init__()

            self.input_size = input_size
            self.hidden_size = hidden_size
            self.batch_first = batch_first

            # Класс реализующий ячейку LSTM
            self.rnn_cell = rnn_cell(self.input_size, self.hidden_size)

        def forward(self, X, state=None):
            """
            При прямом проходе необходимо получить скрытые состояния h_t
            для каждого момента времени. Это нужно, например для задач типа mapu
            """
            # X размера N x L x M
            batch_size = X.shape[0]

```

```

if state is None:
    # Очень важно! По умолчанию тензор будет на CPU и произойдёт оши
    h_t = X.new_zeros((batch_size, self.hidden_size))
    c_t = X.new_zeros((batch_size, self.hidden_size))
else:
    h_t, c_t = state

output = []
for t in range(X.shape[1]):
    if self.batch_first:
        h_t, c_t = self.rnn_cell(X[:,t,:], (h_t, c_t))
    else:
        h_t, c_t = self.rnn_cell(X[t,:,:], (h_t, c_t))
    # Сохраняем новый выход сети
    output.append(h_t)

return torch.stack(output, dim=1), (h_t, c_t)

```

```

In [13]: class Simple_RNN_Classifier(nn.Module):
    """
    Класс классификации с использованием произвольной RNN
    """
    def __init__(self, input_size, hidden_size, num_class, rnn,
                  activation=F.relu, dropout_p=0.5):
        super().__init__()

        # 1. Тут могла быть ваша предобработка данных
        # Но в простой модели её нет

        # 2. Тут рекуррентная часть модели
        # Поступает уже инициализированная рекуррентная сеть
        self.rnn = rnn

        # 3. Тут классификатор
        # Классификатор - два полносвязных слоя с dropout
        self.classifier = nn.Sequential(
            nn.BatchNorm1d(hidden_size),
            nn.Linear(hidden_size, 40),
            nn.Dropout(dropout_p / 2.0),
            activation,
            nn.Linear(40, num_class)
        )

    def forward(self, input):
        # Наш ряд из Dataloader имеет представление (N, M, T)
        # Размер входа в LSTM (N, T, M) -> нужна трансформация
        input = input.swapaxes(1, 2)

        # Проход по LSTM
        try:
            output, hidden = self.rnn(input)
        except Exception:
            output = self.rnn(input)
        # Возвращается h_t для всех моментов t = 1, 2, ..., T
        # Размерность output (N, T, hidden)

```



```

        # Получаем h_T
        output = output[:, -1, :]

        # Проводим классификацию
        output = self.classifier(output)
        return output

device = torch.device('cuda:0') if torch.cuda.is_available() else torch.device

```

Out[13]: device(type='cuda', index=0)

```

In [14]: def print_params(model: nn.Module):
          rnn = model.rnn
          x, y = sum(p.numel() for p in rnn.parameters()), sum(p.numel() for p in
          print(f'RNN содержит {x} обучаемых параметров')
          print(f'Классификатор содержит {y - x} обучаемых параметров')

```

Сравнение различных подходов в RNN

В PyTorch доступно всего 3 модели RNN:

1. Простая RNN (Элмана)
2. LSTM
3. GRU

Посмотрим как обучаются различные модели на нашей задаче.

А именно, рассмотрим графики качества, функции потерь и средней по выборке

нормы градиента $\frac{\partial L_{t+1}}{\partial W_{hh}}, \frac{\partial L_{t+1}}{\partial W_{ih}}$.

Определим среднюю норму градиента по выборке для нашей many-to-one задачи:

Пусть выборка разбита на k одинаковых по размеру батчей $(\hat{X}_1, \dots, \hat{X}_k)$. Тогда средняя норма градиента по параметру ω равняется

$$\sqrt{\frac{1}{k} \sum_{n=1}^k \left\| \frac{\partial L}{\partial W_{hh} \ n} \right\|^2}$$

Здесь $\frac{\partial L}{\partial W_{hh} \ n}$ – оценка градиента по n -ому батчу.

```

In [15]: def plot_losses(train_history, test_history, title):
          plt.subplots(1, 2, figsize=(10,5))
          plt.suptitle(title)
          plt.subplot(1, 2, 1)
          plt.plot(train_history['loss'], label='Train')
          plt.plot(test_history['loss'], label='Test')
          plt.xlabel('Эпоха')
          plt.ylabel('Loss')

```

```

plt.legend(loc='upper right')
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(train_history['acc'], label='Train')
plt.plot(test_history['acc'], label='Test')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy')
plt.legend(loc='upper left')
plt.grid()
plt.tight_layout()
plt.show()

```

```

In [16]: def plot_results(results_train, results_test, title):
    colors = ['b', 'r', 'g']
    plt.subplots(1, 2, sharex=True, figsize=(10,5))
    plt.suptitle(title)

    for idx, (k, train_history) in enumerate(results_train.items()):
        test_history = results_test[k]
        c = colors[idx]

        plt.subplot(1, 2, 1)
        plt.plot(train_history['loss'], label=f'{k} Train', c=c, linestyle='--')
        plt.plot(test_history['loss'], label=f'{k} Test', c=c)
        plt.xlabel('Эпоха')
        plt.ylabel('Loss')
        plt.legend(loc='upper right')
        plt.grid()

        plt.subplot(1, 2, 2)
        plt.plot(train_history['acc'], label=f'{k} Train', c=c, linestyle='--')
        plt.plot(test_history['acc'], label=f'{k} Test', c=c)
        plt.xlabel('Эпоха')
        plt.ylabel('Accuracy')
        plt.legend(loc='upper left')
        plt.grid()

    plt.tight_layout()
    plt.show()

```

```

In [17]: def plot_grads(grad, name):
    #Строим графики для нормы градиента
    plt.figure(figsize=(10, 5))
    plt.title(r'Средняя норма градиента, ' + name, fontsize=14)
    plt.xlabel('Номер эпохи', fontsize=14)
    plt.ylabel('Средняя норма градиента', fontsize=14)
    plt.plot(grad[:, 1], label=r'$\left|\frac{\partial L}{\partial W_{hh}}\right|$')
    plt.plot(grad[:, 0], label=r'$\left|\frac{\partial L}{\partial W_{ih}}\right|$')
    plt.plot(np.ones(200), label=r'$\mathrm{norm} = 1$')

    plt.legend(loc='upper right', fontsize=14)
    plt.grid()
    plt.tight_layout()
    plt.show()

```

```
In [18]: def get_exps(model, optimizer, scheduler, title, num_epoch=200, max_norm=None):
# Проводим обучение и строим графики loss, accuracy
train_history, test_history, grad_history = train(model, device, train_loader,
num_epoch, optimizer, scheduler, max_norm, track_gradients=True)
plot_losses(train_history, test_history, title)
return train_history, test_history, grad_history
```

Мы сравним между собой разные версии RNN. Обратите внимание, что случайное угадывания имеет точность примерно 0.17. Так как мы будем исследовать архитектуры, то зафиксируем все остальные параметры обучения.

```
In [19]: SEED = 42
BS = 10
INPUT_SIZE = 24
NUM_CLASSES = 6
LR = 5e-4
STEP_SIZE = 100
GAMMA = 0.9
NUM_EPOCHES = 200
```

RNN Элмана

Попытаемся обучить RNN с размерностью скрытого представления 230. В этом случае модель имеет примерно то же число параметров, что и LSTM с скрытой размерностью 100.

```
In [20]: def setup_loaders():
g = torch.Generator()
g.manual_seed(SEED)
test_loader = DataLoader(test_dataset, batch_size=BS, worker_init_fn=seed_worker)
train_loader = DataLoader(train_dataset, batch_size=BS, drop_last=True,
generator=g)
return test_loader, train_loader
```

```
In [21]: set_global_seed(SEED) # Не забываем!
test_loader, train_loader = setup_loaders()

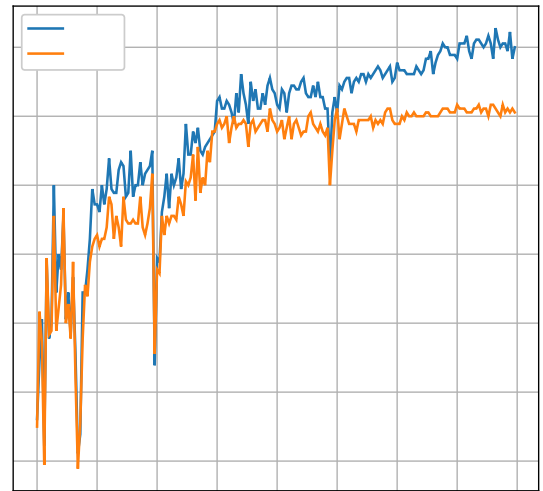
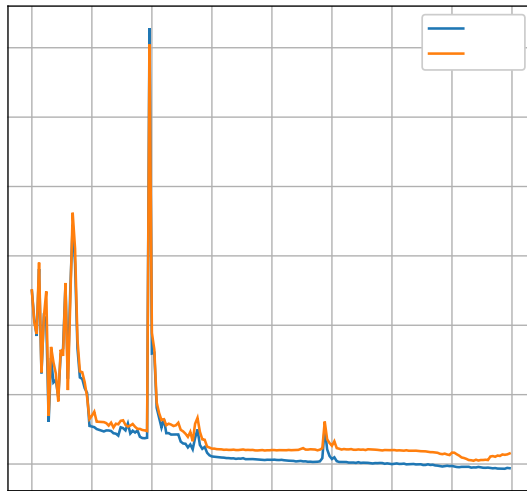
rnn = nn.RNN(INPUT_SIZE, 230, batch_first=True)
model = Simple_RNN_Classifier(INPUT_SIZE, 230, NUM_CLASSES, rnn, nn.ReLU())

print_params(model)

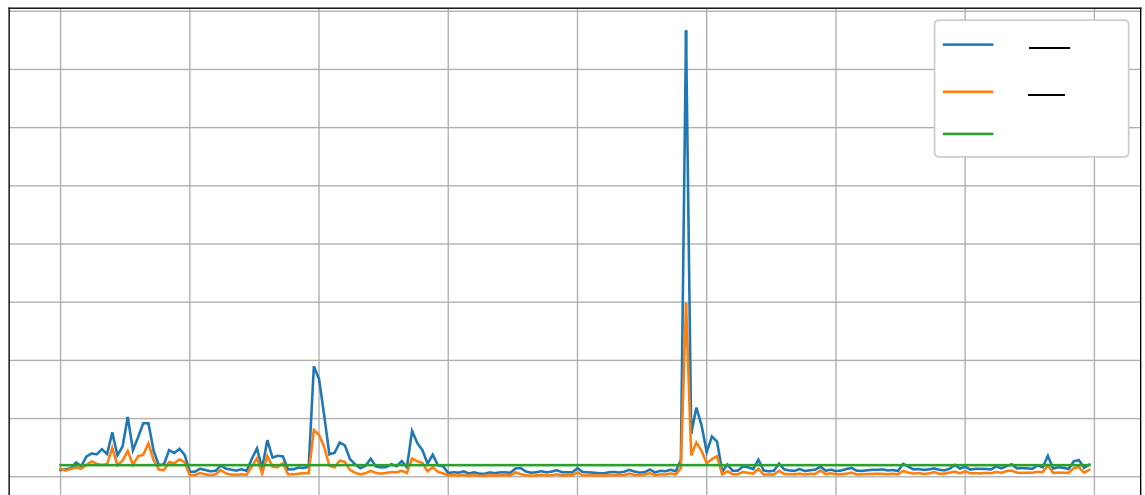
optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

_, _, grad_history_RNN = get_exps(model, optimizer, scheduler, 'Обычная RNN')
```

RNN содержит 58880 обучаемых параметров
Классификатор содержит 9946 обучаемых параметров
0%| | 0/200 [00:00<?, ?it/s]



```
In [22]: plot_grads(grad_history_RNN, 'Обычная RNN')
```



```
In [23]: delete_model(model)
delete_model(rnn)
```

Облегчённая RNN

Уменьшим размерность скрытого слоя.

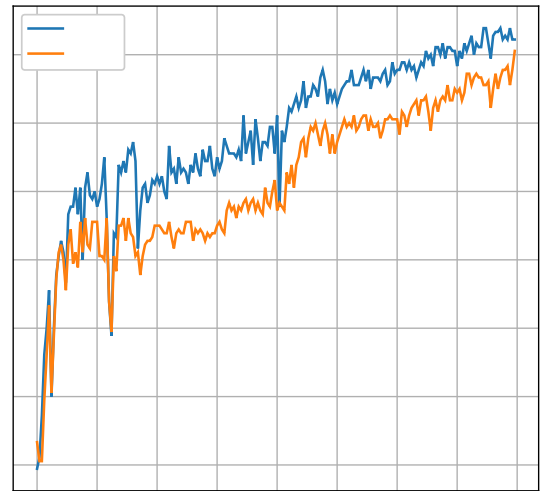
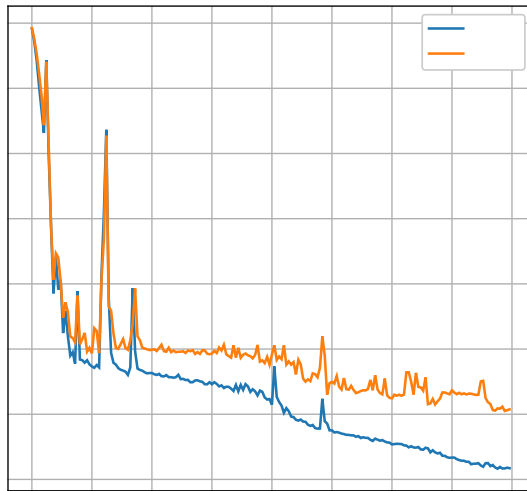
```
In [24]: set_global_seed(SEED)
test_loader, train_loader = setup_loaders()

rnn = nn.RNN(INPUT_SIZE, 80, batch_first=True)
model = Simple_RNN_Classifier(INPUT_SIZE, 80, NUM_CLASSES, rnn, nn.ReLU()).t
print_params(model)
```

```
optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

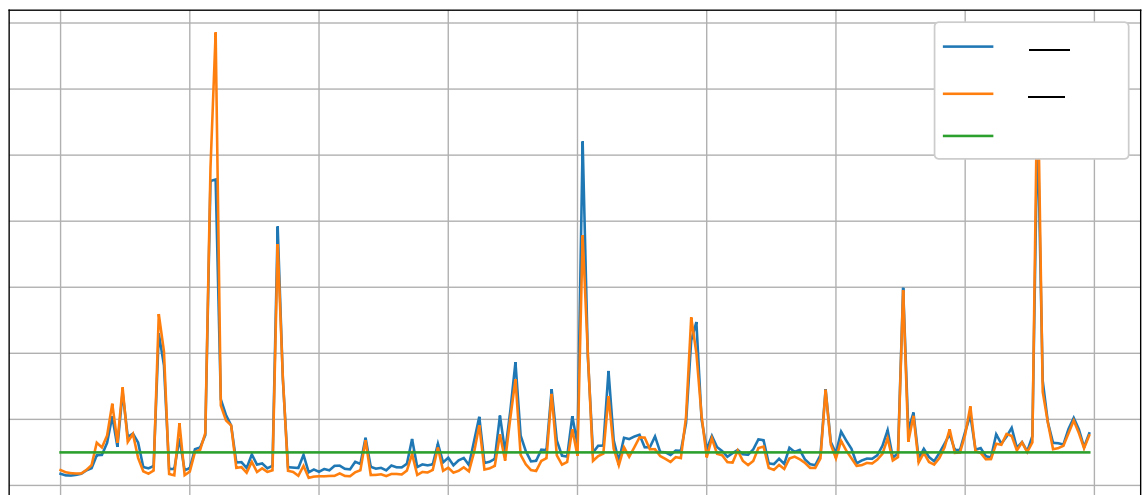
train_history_RNN, test_history_RNN, grad_history_RNN = get_exps(model, opti
```

RNN содержит 8480 обучаемых параметров
 Классификатор содержит 3646 обучаемых параметров
 0%| | 0/200 [00:00<?, ?it/s]



Обратите внимание, что обучение стало стабильнее как на train так и на test. Кроме того, переобучение стало меньше.

```
In [25]: plot_grads(grad_history_RNN, 'Облегчённая RNN')
```



```
In [26]: delete_model(model)
delete_model(rnn)
```

Handmade LSTM

```

In [27]: set_global_seed(SEED)
test_loader, train_loader = setup_loaders()

lstm = My_LSTM_Layer(INPUT_SIZE, 100, nn.LSTMCell, batch_first=True)
model = Simple_RNN_Classifier(INPUT_SIZE, 100, NUM_CLASSES, lstm, nn.ReLU())

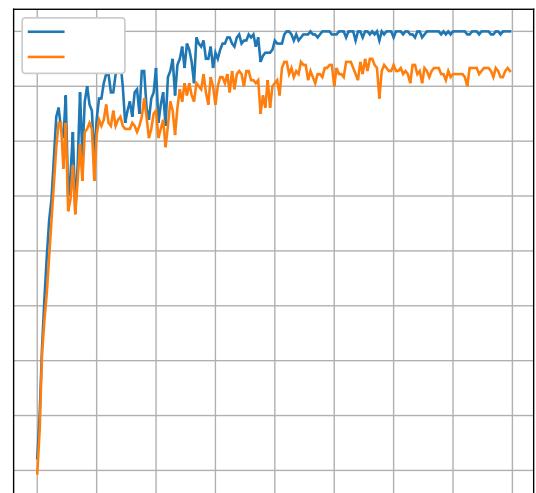
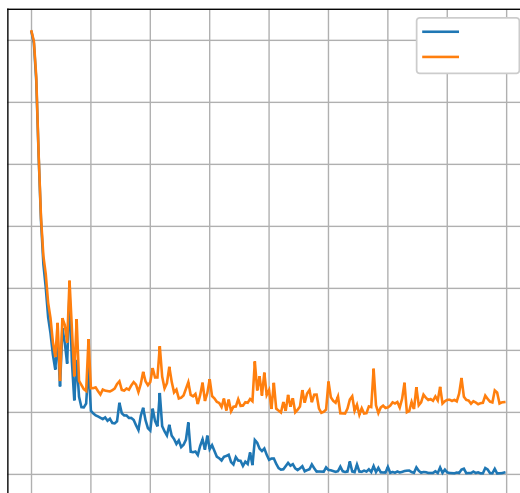
print_params(model)

optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

_, _, grad_history_HM_LSTM = get_exps(model, optimizer, scheduler, 'Handmade

```

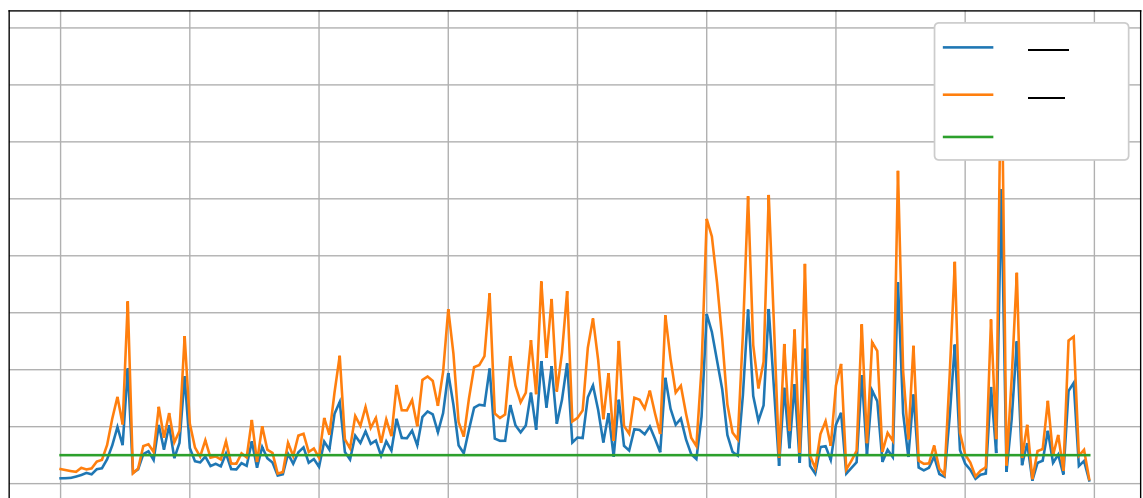
RNN содержит 50400 обучаемых параметров
Классификатор содержит 4486 обучаемых параметров
0%| | 0/200 [00:00<?, ?it/s]



```

In [28]: plot_grads(grad_history_HM_LSTM, 'Handmade LSTM')

```



```
In [29]: delete_model(model)
delete_model(lstm)
```

Модель показало сильно лучше качество (1 и 0.9 ассурасу на train и test), однако, наша реализация для обучения потребовала 1.5 минуты, вместо 15 секунд. Посмотрим реализацию PyTorch.

LSTM

```
In [30]: set_global_seed(SEED)
test_loader, train_loader = setup_loaders()

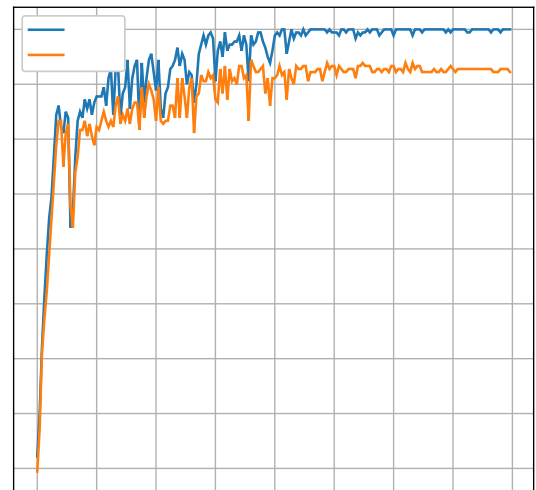
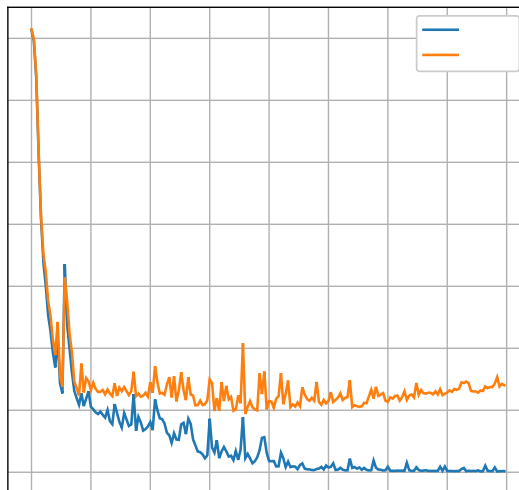
lstm = nn.LSTM(INPUT_SIZE, 100, batch_first=True)
model = Simple_RNN_Classifier(INPUT_SIZE, 100, NUM_CLASSES, lstm, nn.ReLU())

print_params(model)

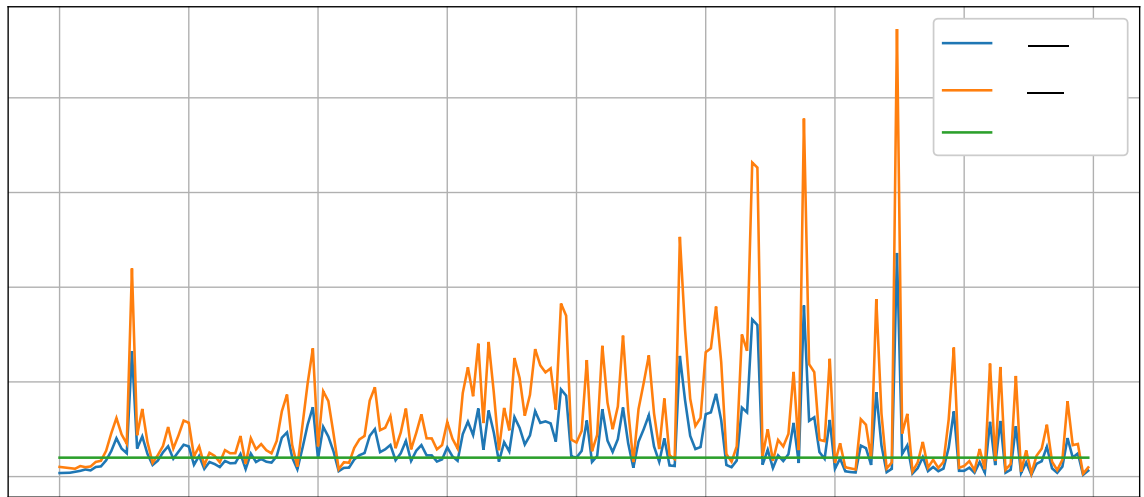
optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

train_history_LSTM, test_history_LSTM, grad_history_LSTM = get_exps(model, c
```

RNN содержит 50400 обучаемых параметров
Классификатор содержит 4486 обучаемых параметров
0%| | 0/200 [00:00<?, ?it/s]



```
In [31]: plot_grads(grad_history_LSTM, 'PyTorch LSTM')
```



```
In [32]: delete_model(model)
delete_model(lstm)
```

Реализация PyTorch работает сравнимо с RNN по времени.

GRU

```
In [33]: set_global_seed(SEED)
test_loader, train_loader = setup_loaders()

gru = nn.GRU(INPUT_SIZE, 100, batch_first=True)
model = Simple_RNN_Classifier(INPUT_SIZE, 100, NUM_CLASSES, gru, nn.ReLU()).
print_params(model)

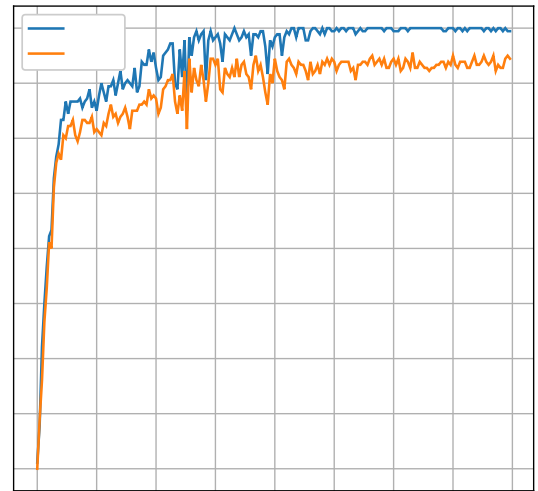
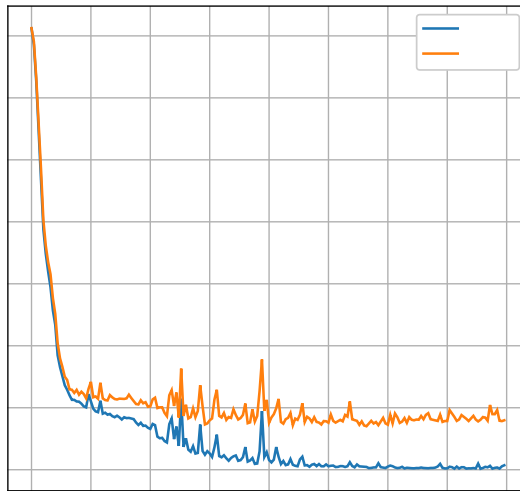
optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

train_history_GRU, test_history_GRU, grad_history_GRU = get_exps(model, opti
```

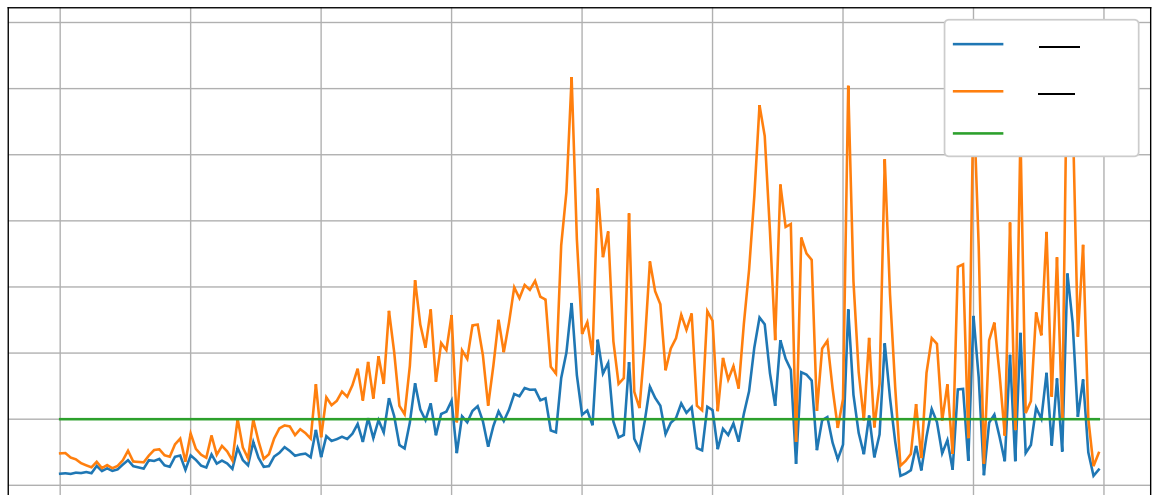
RNN содержит 37800 обучаемых параметров

Классификатор содержит 4486 обучаемых параметров

0%| | 0/200 [00:00<?, ?it/s]



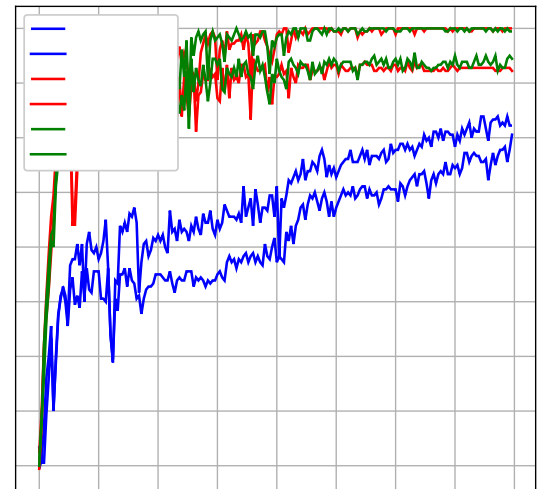
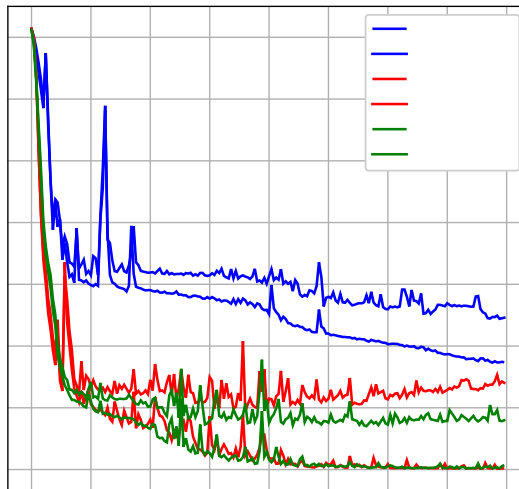
```
In [34]: plot_grads(grad_history_GRU, 'GRU')
```



```
In [35]: delete_model(model)
delete_model(gru)
```

```
In [36]: results_train = {
    'RNN': train_history_RNN,
    'LSTM': train_history_LSTM,
    'GRU': train_history_GRU
}
results_test = {
    'RNN': test_history_RNN,
    'LSTM': test_history_LSTM,
    'GRU': test_history_GRU
}

plot_results(results_train, results_test, 'Сравнение базовых моделей')
```



Gradient clipping

Выше мы пронаблюдали, что часто градиенты взрываются, а значения функции потерь в последствии скачут. Это может мешать процессу оптимизации.

In [37]: MAX_NORM = 5

```
In [38]: set_global_seed(SEED)
test_loader, train_loader = setup_loaders()

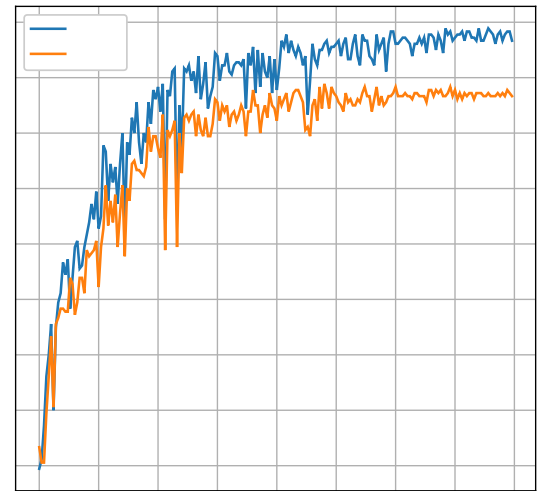
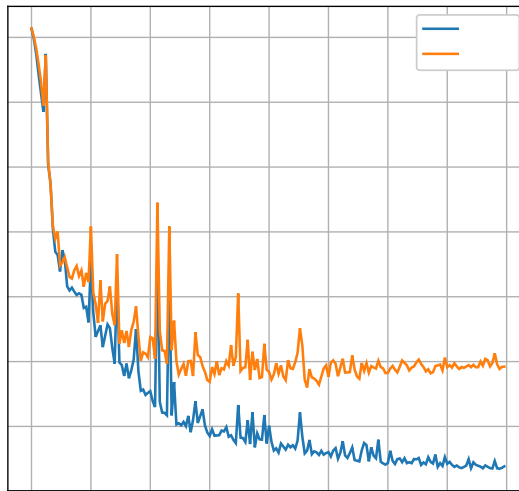
rnn = nn.RNN(INPUT_SIZE, 80, batch_first=True)
model = Simple_RNN_Classifier(INPUT_SIZE, 80, NUM_CLASSES, rnn, nn.ReLU()).t

print_params(model)

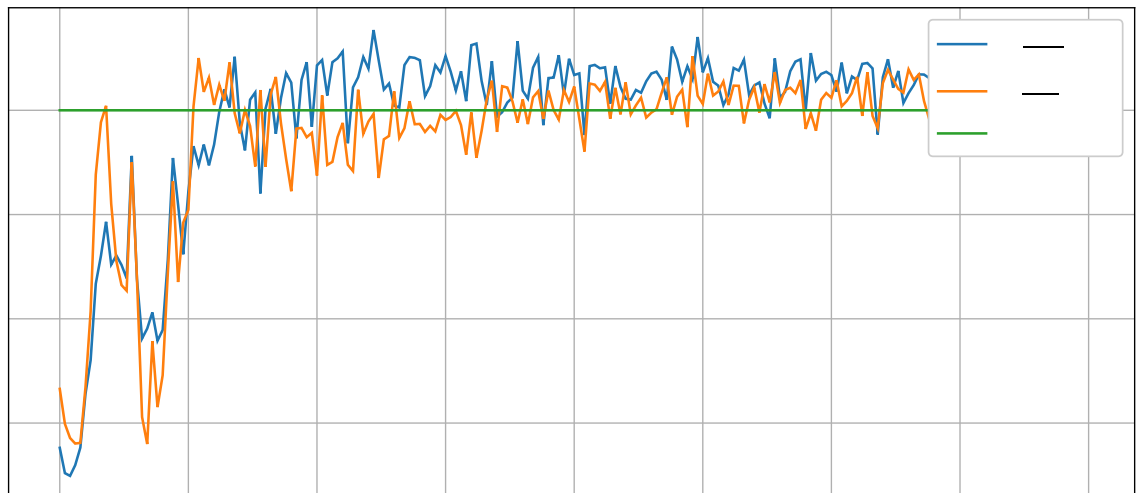
optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

train_history_clip, test_history_clip, grad_history_clip = get_exps(
    model,
    optimizer,
    scheduler,
    'Облегчённая RNN с gradient clipping',
    num_epoch=NUM_EPOCHES,
    max_norm=MAX_NORM
)
```

RNN содержит 8480 обучаемых параметров
 Классификатор содержит 3646 обучаемых параметров
 0%| | 0/200 [00:00<?, ?it/s]



```
In [39]: plot_grads(grad_history_clip, 'RNN c gradient clipping')
```



Как можно заметить, модель стала меньше флуктуировать, однако она все еще уступает другим моделям. Возможно дело в том, что градиент достаточно мал уже в самом начале или в том, что мы не решили проблему затухания в принципе. Попробуем исправить это ортогональной инициализацией

```
In [40]: delete_model(model)
delete_model(rnn)
```

Gradient clipping + orthogonal initialization

Инициализация весов ортогональной матрицей весов позволяет сохранять норму активаций при проходе через слой, что привносит стабильности в обучение. Попробуем инициализировать параметры нашего классификатора подобным образом

```

In [41]: set_global_seed(SEED)
test_loader, train_loader = setup_loaders()

rnn = nn.RNN(INPUT_SIZE, 80, batch_first=True)
model = Simple_RNN_Classifier(INPUT_SIZE, 80, NUM_CLASSES, rnn, nn.ReLU()).t

# Ортогональная инициализация
for p in model.parameters():
    if len(p.shape) == 2:
        nn.init.orthogonal_(p)

print_params(model)

optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

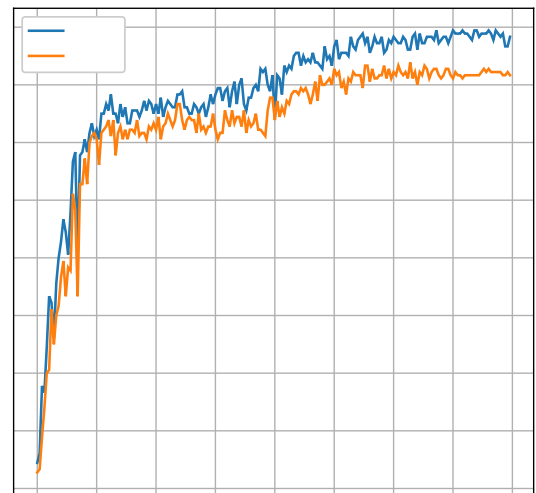
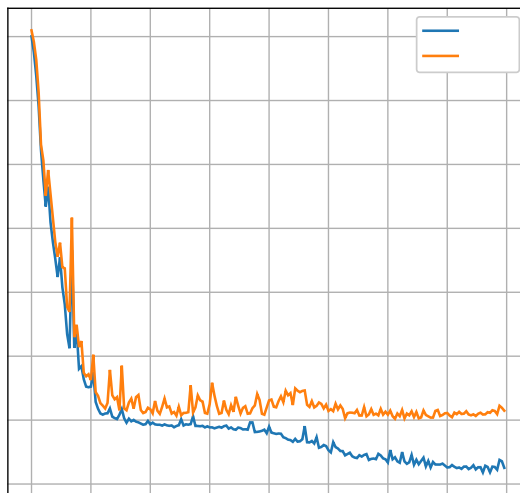
train_history_init_clip, test_history_init_clip, grad_history_init_clip = ge
num_epoch=NUM_EPOCHES, max_norm=MAX_

```

RNN содержит 8480 обучаемых параметров

Классификатор содержит 3646 обучаемых параметров

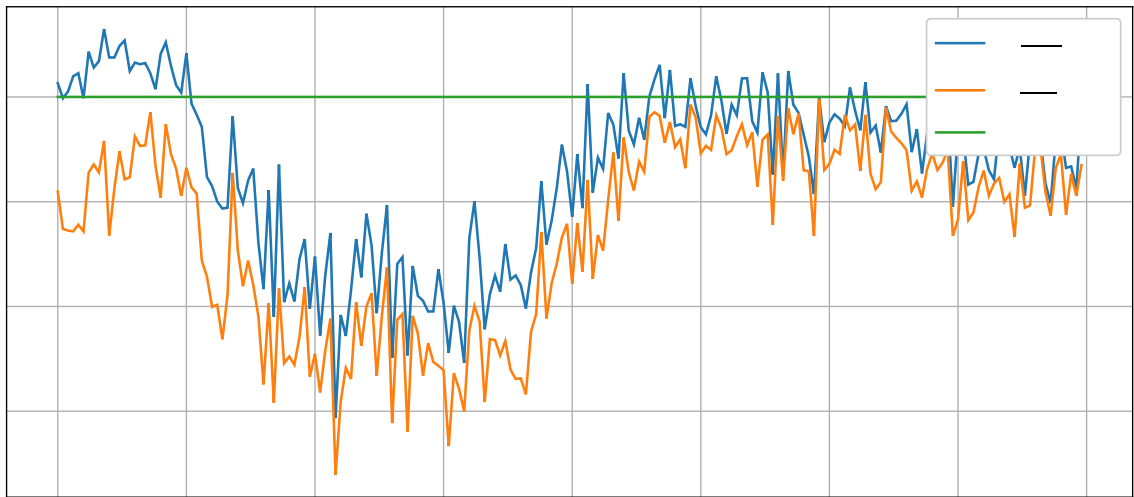
0%| | 0/200 [00:00<?, ?it/s]



```

In [42]: plot_grads(grad_history_init_clip, 'RNN с gradient clipping и orthogonal ini

```



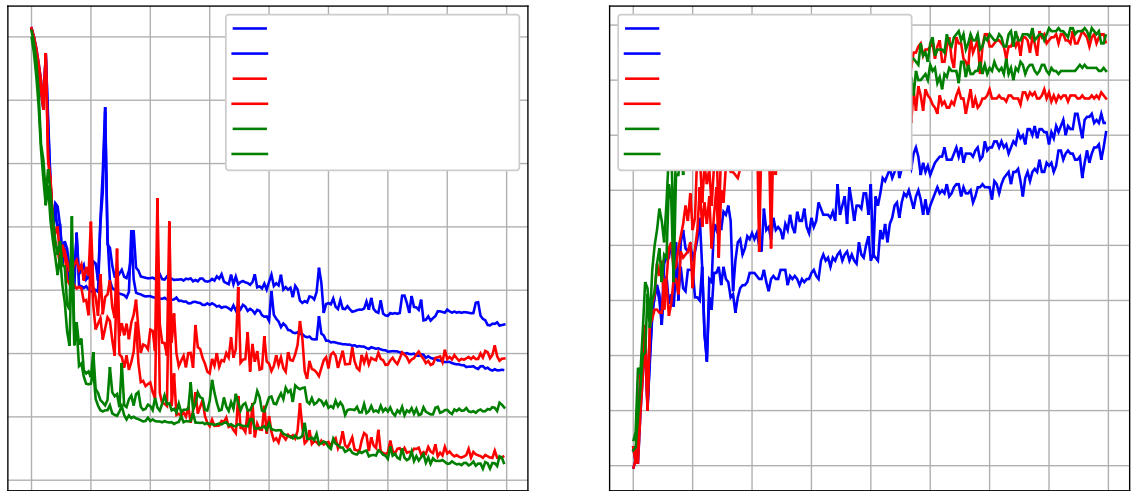
Как итог можно заявить, что модель классическая RNN модель обучается, но очень нестабильно. Что важно, качество на train и test примерно одинаковое, т.е. скачет общее качество модели. Применение специальных техник в данном случае дало существенное улучшение в качестве.

В то же время использование LSTM позволяет получить отличное качество на данном примере.

```
In [43]: delete_model(model)
         delete_model(rnn)
```

```
In [44]: results_train = {
         'RNN': train_history_RNN,
         'Gradient clipping': train_history_clip,
         'Init, Gradient clipping': train_history_init_clip
       }
       results_test = {
         'RNN': test_history_RNN,
         'Gradient clipping': test_history_clip,
         'Init, Gradient clipping': test_history_init_clip
       }

       plot_results(results_train, results_test, 'Сравнение RNN с модификациями')
```



Bidirectional и Stacked LSTM

Допустим вы хотите применить BiLSTM и сделать модель с 3 LSTM друг поверх друга. Тогда в базовой версии PyTorch вы получите 6 LSTM – 3 пары. Более того, пусть `input_size = 11`, `hidden_size = 100`, тогда все пары LSTM будут иметь матрицы W_{hh} размера 400×100 , выход будет иметь размерность 200 (вектор размерности 100 от первой модели и 100 от второй). Однако матрицы W_{ih} по парам будут иметь размеры 400×11 , 400×200 , 400×200 соответственно.

```
In [45]: set_global_seed(SEED)
test_loader, train_loader = setup_loaders()

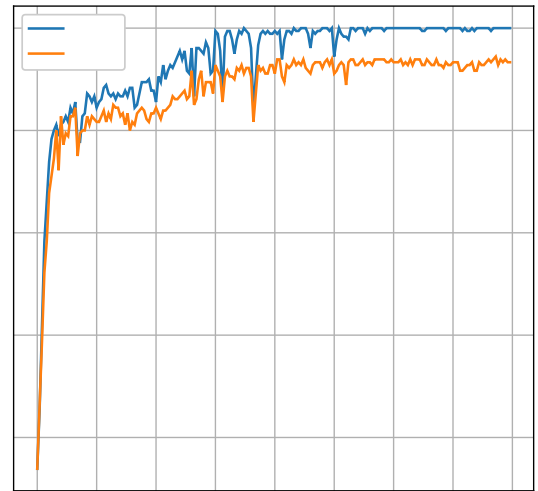
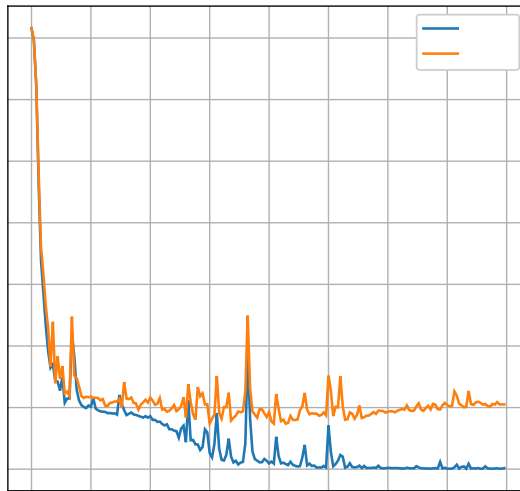
lstm = nn.LSTM(INPUT_SIZE, 100, batch_first=True, num_layers=2, bidirectional=True)
model = Simple_RNN_Classifier(INPUT_SIZE, 2 * 100, NUM_CLASSES, lstm, nn.ReLU)

print_params(model)

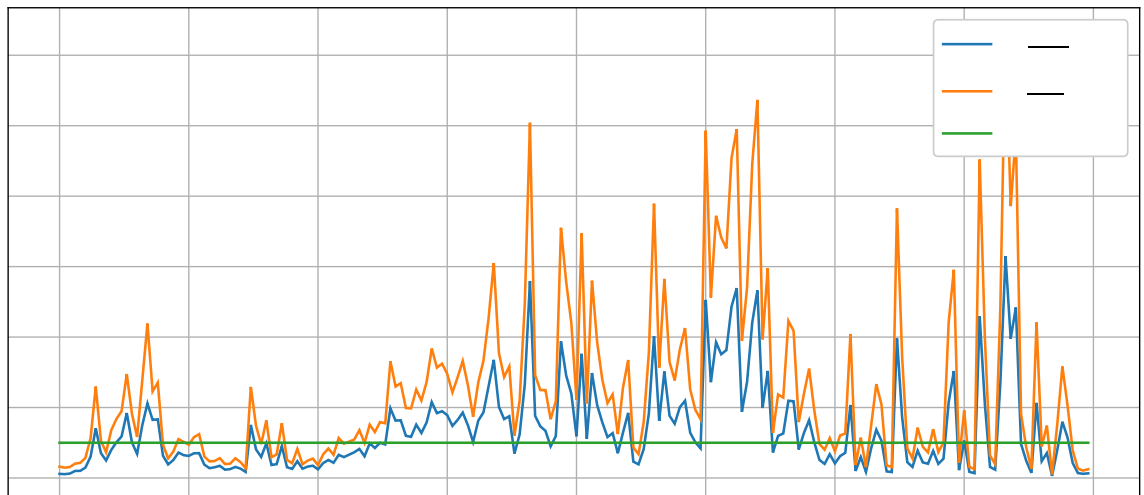
optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

train_history_BiLSTM, test_history_BiLSTM, grad_history_BiLSTM = get_exps(model, train_loader, test_loader, optimizer, scheduler)
```

```
RNN содержит 342400 обучаемых параметров
Классификатор содержит 8686 обучаемых параметров
0%|          | 0/200 [00:00<?, ?it/s]
```



```
In [46]: plot_grads(grad_history_BiLSTM, 'Doubled BiLSTM')
```



Большая сложность модели привела к более гладкому обучению. Графики флуктуируют намного меньше, чем для обычной LSTM.

На данной задаче LSTM показала [SOTA качество](#) (нужно учесть, что задача NATOPS максимально простая)

```
In [47]: delete_model(model)
delete_model(lstm)
```

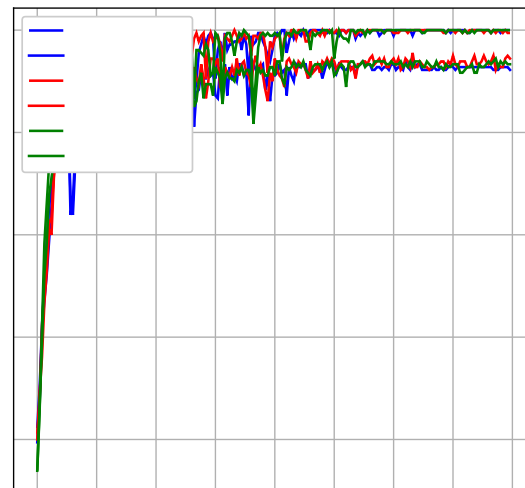
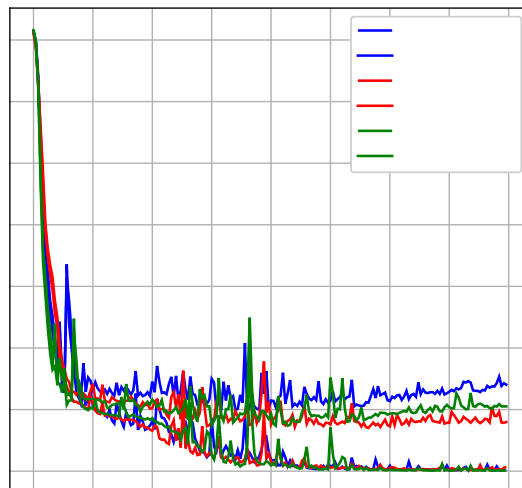
```
In [48]: results_train = {
    'LSTM': train_history_LSTM,
    'GRU': train_history_GRU,
    'BiLSTM': train_history_BiLSTM,
}
results_test = {
```

```

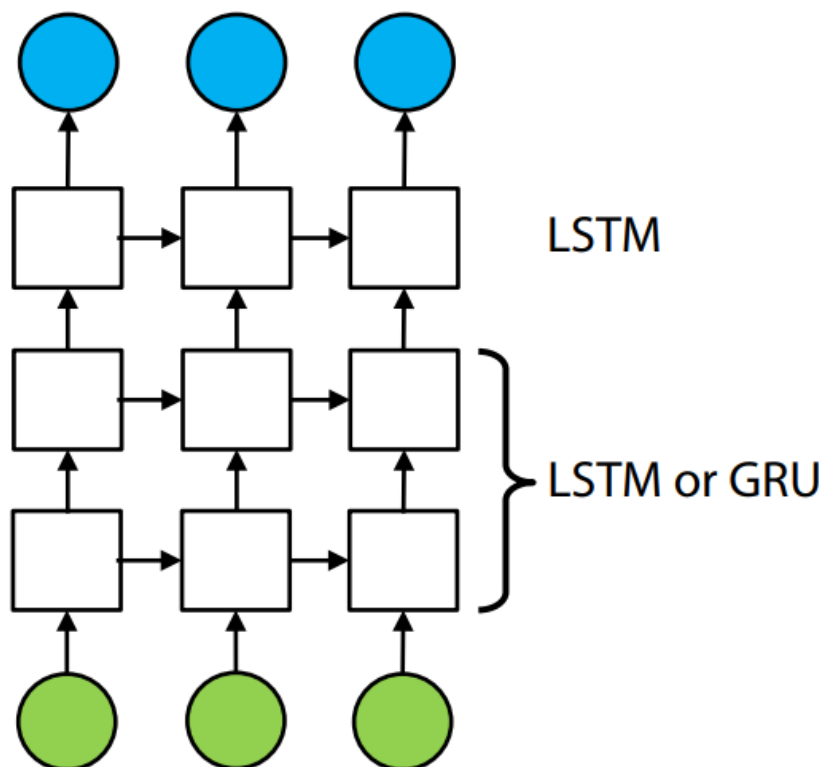
'LSTM': test_history_LSTM,
'GRU': test_history_GRU,
'BiLSTM': test_history_BiLSTM,
}

plot_results(results_train, results_test, 'Сравнение лучших моделей')

```



Общее правило по использованию LSTM и GRU, полученное коммьюнити эмпирическим путем: последней рекуррентностью стоит использовать LSTM, а перед ней можно как и LSTM, так и GRU.



Тестирование 'памяти' рекуррентных нейронных сетей

Для проверки способности вашей модели запоминать информацию на долгий период стандартно используется набор PSMNIST. (Хотя он несколько подвергается критике)

PermutedSequentialMNIST – датасет получаемый перестановкой пикселей в случайном порядке (фиксирован и един для всех картинок). Мы можем смотреть на объект как на последовательность длины $28 \cdot 28$ (каждый элемент имеет размерность 1), либо как на последовательность длины 28 (каждый элемент имеет размерность 28).

```
In [49]: ROOT = './data'
# ROOT = './data' # Если Google Colab, то нужно раскомментировать строчку

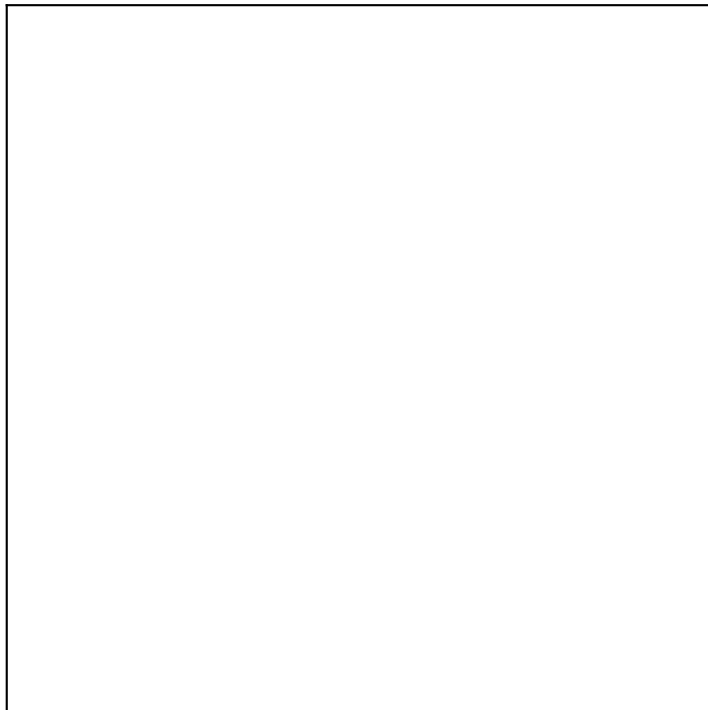
NUM_CLASSES = 10
BS = 512
```

```
In [50]: # Загрузка обычного MNIST
transform=T.Compose([T.ToTensor(), T.Normalize((0.1307,), (0.3081,))])
train_dataset_MNIST = torchvision.datasets.MNIST(ROOT, train=True, download=
test_dataset_MNIST = torchvision.datasets.MNIST(ROOT, train=False, download=

train_loader = torch.utils.data.DataLoader(train_dataset_MNIST, batch_size=E

plt.imshow(next(iter(train_loader))[0][1][0].cpu().numpy(), cmap='Greys')
next(iter(train_loader))[0].cpu().numpy().shape
```

```
Out[50]: (512, 1, 28, 28)
```



Для получения PSMNIST достаточно добавить преобразование перестановки пикселей. Отметим, что все преобразования делаются до перевода данных на GPU.

```
In [51]: class Image_permutation():
        """
        Перестановка пикселей в картинке
        """
        def __init__(self, c, h, w, permutation=None, flatten_flag=False):
            """
            c, h, w - параметры картинки
            permutation - перестановка, если None - генерируем новую
            flatten_flag - если False, то возвращаем объект вида c x h x w
                        если True, то c * h * w x 1
            """
            self.h = h
            self.w = w
            if permutation is not None:
                self.permutation = permutation
            else:
                self.permutation = torch.randperm(c * h*w)
            self.flatten_flag = flatten_flag

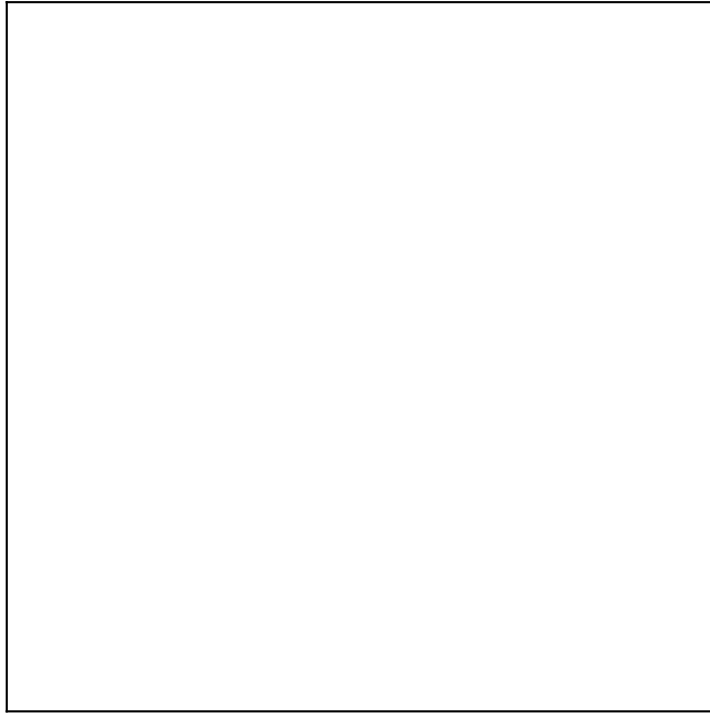
        def __call__(self, object):
            if self.flatten_flag:
                return object.flatten()[self.permutation].reshape(-1, 1)
            else:
                return object.flatten()[self.permutation].reshape((c, self.h, se
```

```
In [52]: # Загрузка MNIST
transform=T.Compose([
    T.ToTensor(),
    T.Normalize((0.1307,), (0.3081,)),
    Image_permutation(1, 28, 28, flatten_flag=True) # Добавляем преобраз
])
train_dataset_PMNIST = torchvision.datasets.MNIST(ROOT, train=True, transform=transform)
test_dataset_PMNIST = torchvision.datasets.MNIST(ROOT, train=False, transform=transform)

train_loader = DataLoader(train_dataset_PMNIST, batch_size=BS)

plt.imshow(next(iter(train_loader))[0][1].cpu().numpy().reshape(28, 28), cmap=cm.gray)
next(iter(train_loader))[0].cpu().numpy().shape
```

Out[52]: (512, 784, 1)



```
In [53]: class MNIST_LSTM_Classifier(nn.Module):
    def __init__(self, input_size, hidden_size, n_classes, rnn, activation,
                 super().__init__())
        self.pixel_wise = pixel_wise
        self.rnn = rnn

        # Классификатор - простой линейный слой
        self.classifier = nn.Sequential(
            nn.BatchNorm1d(hidden_size), nn.Linear(hidden_size, 32), activation
        )

    def forward(self, input):
        # Размер входа в LSTM (N, L, M)
        N = input.shape[0]
        if self.pixel_wise:
            # Наш ряд из Dataloader имеет представление (N, L, 1)
            input = input.reshape(N, -1, 1)
        else:
            # Наш ряд из Dataloader имеет представление (N, C, H, W)
            L = input.shape[2]
            input = input.reshape(N, L, -1)
        # Проход по LSTM
        output, hidden = self.rnn(input)

        # Размерность output (N, L, hidden)
        # Получаем h_L
        output = output[:, -1, :]

        # Проводим классификацию
        output = self.classifier(output)
        return output
```

Обучим LSTM, GRU, RNN (Elman) и сравним их способность запоминать длинные последовательности.

```
In [54]: def setup_loaders():
    g = torch.Generator()
    g.manual_seed(SEED)
    test_loader = DataLoader(test_dataset, batch_size=BS, worker_init_fn=seed_worker, shuffle=False)
    train_loader = DataLoader(train_dataset, batch_size=BS, drop_last=True, worker_init_fn=seed_worker, shuffle=True)

    return test_loader, train_loader
```

```
In [55]: from time import time

def get_exps_PSMNIST(rnn, in_size, h_size, pixel_wise=False, n_epoch=60, lr=0.001, gamma=0.95):
    set_global_seed(SEED)
    test_loader, train_loader = setup_loaders()

    rnn = rnn(in_size, h_size, batch_first=True)
    model2 = MNIST_LSTM_Classifier(in_size, h_size, NUM_CLASSES, rnn, nn.ReLU, nn.Dropout)

    print_params(model2)

    optimizer = optim.Adam(model2.parameters(), lr=lr)
    scheduler = StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

    t_start = time()
    train_history, test_history = train(model2, device, train_loader, test_loader, n_epoch, optimizer, scheduler)
    t_end = time()
    print(f'Для обучения {n_epoch} эпох потребовалось {(t_end - t_start) / 60} минут')

    delete_model(model2)
    delete_model(rnn)

    return train_history, test_history
```

Посмотрим как модели себя ведут на MNIST, если рассматривать его как последовательность пикселей

1. Длина последовательности: $L = H \cdot W \cdot C = 28 \cdot 28 \cdot 1$
2. Размерность проекции: $M = 1$

```
In [56]: NUM_EPOCHES = 60
test_dataset = test_dataset_PMNIST
train_dataset = train_dataset_PMNIST
```

```
In [57]: train_history_LSTM, test_history_LSTM = get_exps_PSMNIST(nn.LSTM, 1, 52, pixel_wise=True, n_epoch=NUM_EPOCHES, lr=0.001, gamma=0.95)
```

RNN содержит 11440 обучаемых параметров
Классификатор содержит 2130 обучаемых параметров
0% | 0/60 [00:00<?, ?it/s]
Для обучения 60 эпох потребовалось 22.31 минут

```
In [58]: train_history_GRU, test_history_GRU = get_exps_PSMNIST(nn.GRU, 1, 60, pixel_
```

RNN содержит 11340 обучаемых параметров
Классификатор содержит 2402 обучаемых параметров
0%| | 0/60 [00:00<?, ?it/s]
Для обучения 60 эпох потребовалось 22.02 минут

```
In [59]: train_history_RNN, test_history_RNN = get_exps_PSMNIST(nn.RNN, 1, 100, pixel_
```

RNN содержит 10300 обучаемых параметров
Классификатор содержит 3762 обучаемых параметров
0%| | 0/60 [00:00<?, ?it/s]
Для обучения 60 эпох потребовалось 21.16 минут

Теперь посмотрим как модели себя ведут на обычном MNIST

1. Длина последовательности: $L = H = 28$
2. Размерность проекции: $M = W \cdot C = 28 \cdot 1$

```
In [60]: NUM_EPOCHES = 20  
test_dataset = test_dataset_MNIST  
train_dataset = train_dataset_MNIST
```

```
In [61]: train_history_LSTM_MNIST, test_history_LSTM_MNIST = get_exps_PSMNIST(nn.LSTM
```

RNN содержит 21600 обучаемых параметров
Классификатор содержит 2402 обучаемых параметров
0%| | 0/20 [00:00<?, ?it/s]
Для обучения 20 эпох потребовалось 5.81 минут

```
In [62]: train_history_GRU_MNIST, test_history_GRU_MNIST = get_exps_PSMNIST(nn.GRU, 2
```

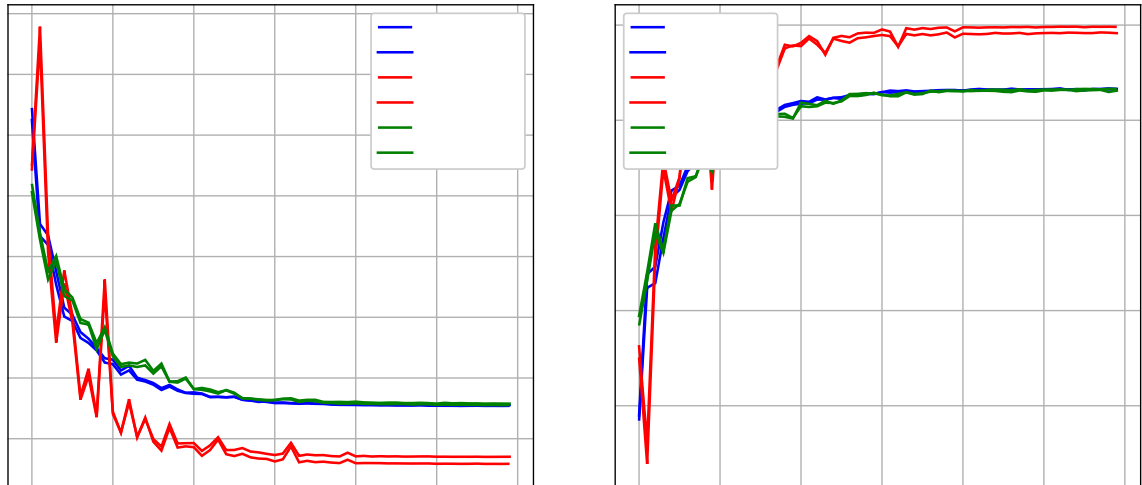
RNN содержит 16200 обучаемых параметров
Классификатор содержит 2402 обучаемых параметров
0%| | 0/20 [00:00<?, ?it/s]
Для обучения 20 эпох потребовалось 6.11 минут

```
In [63]: train_history_RNN_MNIST, test_history_RNN_MNIST = get_exps_PSMNIST(nn.RNN, 2
```

RNN содержит 15400 обучаемых параметров
Классификатор содержит 4102 обучаемых параметров
0%| | 0/20 [00:00<?, ?it/s]
Для обучения 20 эпох потребовалось 5.73 минут

```
In [64]: results_train = {  
    'LSTM': train_history_LSTM,  
    'GRU': train_history_GRU,  
    'RNN': train_history_RNN  
}  
  
results_test = {  
    'LSTM': test_history_LSTM,  
    'GRU': test_history_GRU,  
    'RNN': test_history_RNN  
}
```

```
plot_results(results_train, results_test, r'Сравнение при $L=784$, $M=1$')
```

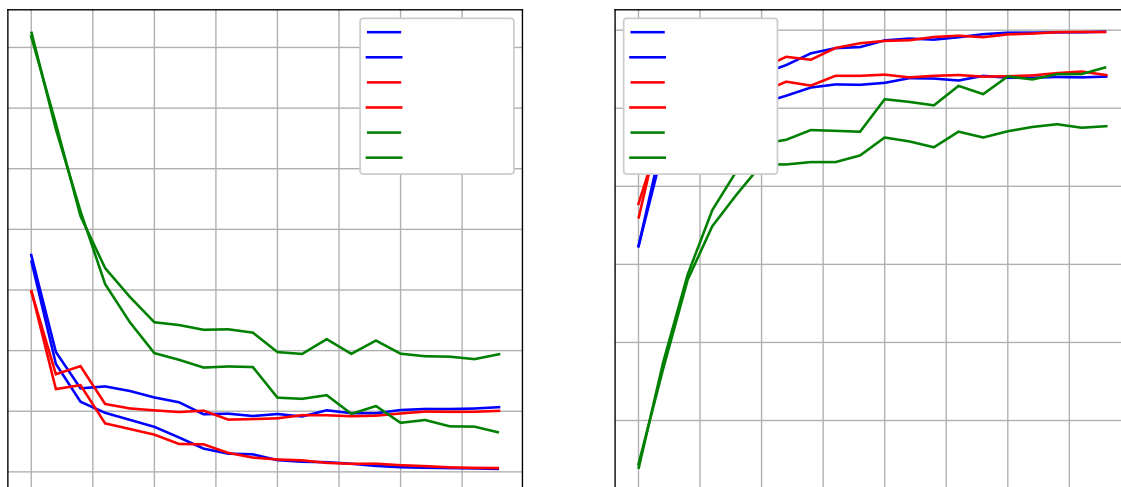


Не стоит удивляться, что LSTM может работать не лучше классической RNN. Это может быть обусловлено тем, что последовательности в PMNIST случайны и для них может быть не нужны некоторые виды долговечной памяти. Разницу между GRU и LSTM исследовали в работе [An Empirical Exploration of Recurrent Network Architectures](#), примерно 2500 цитирований. Авторы пишут, что инициализация forget gate может значительно влиять на качество LSTM, кроме того, LSTM бывает хуже GRU вне задач языкового моделирования.

```
In [65]: results_train = {
    'LSTM': train_history_LSTM_MNIST,
    'GRU': train_history_GRU_MNIST,
    'RNN': train_history_RNN_MNIST
}

results_test = {
    'LSTM': test_history_LSTM_MNIST,
    'GRU': test_history_GRU_MNIST,
    'RNN': test_history_RNN_MNIST
}

plot_results(results_train, results_test, r'Сравнение при $L=28$, $M=28$')
```



Действительно, даже для таких длинных последовательностей наши модели оказались применимы, что говорит о том, что с памятью у них все в порядке. Отметим, что время обучения на PSMNIST в 2 раза привысило время обучения на MNIST. Это вызвано недостатком памяти GPU. Другая проблема заключается в постоянном использовании питоновских циклов при проходе по RNN – отсутствию параллельности. Этот аспект в целом является главной проблемой рекуррентных сетей.

Рассмотрим следующую таблицу, сравнивающую теоретическую эффективность моделей, работающих с последовательностями. Приводится в статье [Efficiently Modeling Long Sequences with Structured State Spaces](#), примерно 4500 цитирований.

Table 1: Complexity of various sequence models in terms of sequence length (L), batch size (B), and hidden dimension (H); tildes denote log factors. Metrics are parameter count, training computation, training space requirement, training parallelizability, and inference computation (for 1 sample and time-step). For simplicity, the state size N of S4 is tied to H . Bold denotes model is theoretically best for that metric. Convolutions are efficient for training while recurrence is efficient for inference, while SSMs combine the strengths of both.

	Convolution ³	Recurrence	Attention	S4
Parameters	LH	H^2	H^2	H^2
Training	$\tilde{L}H(B + H)$	BLH^2	$B(L^2H + LH^2)$	$BH(\tilde{H} + \tilde{L}) + B\tilde{L}H$
Space	BLH	BLH	$B(L^2 + HL)$	BLH
Parallel	Yes	No	Yes	Yes
Inference	LH^2	H^2	$L^2H + H^2L$	H^2

Вывод в рекуррентной модели на каждом шаге производится за H^2 , т.е. не зависит от длины последовательности.

Время обучения также асимптотически линейно от длины последовательности. Однако нужно понимать, что в данном случае скорость RNN по длине является скоростью перехода по питоновским циклам, а для свёрточной сети связана с длиной свёртки, т.е. в первом случае отсутствует возможность параллельного вычисления. Поэтому на практике при большой длине L RNN сильно уступает в скорости свёрточной модели на этапе обучения (в качестве примера свёрточной архитектуры для обработки последовательностей рекомендуется посмотреть [InceptionTime](#). Более того,

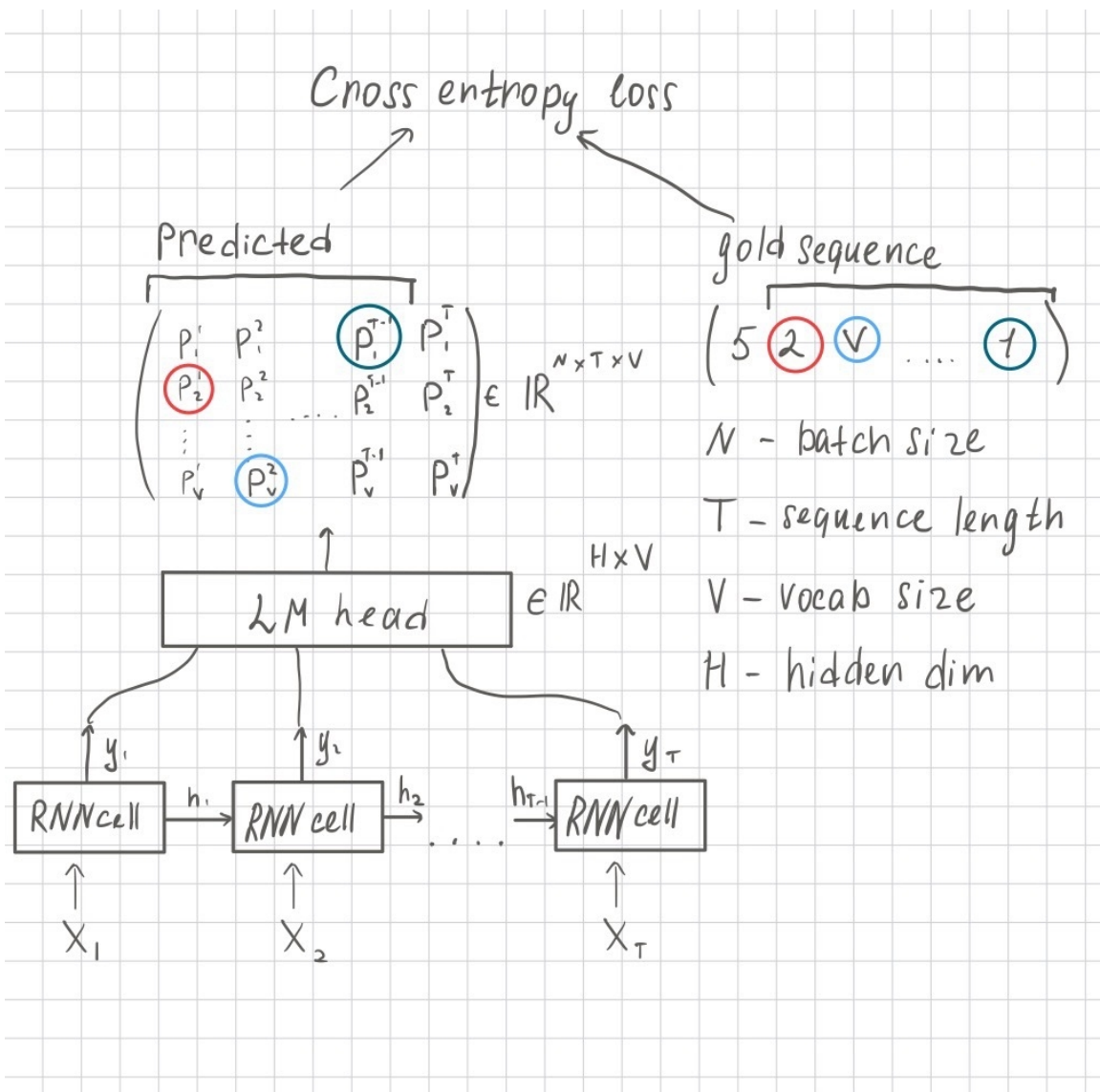
современные архитектуры на основе механизма внимания также могут учиться быстрее рекуррентных сетей за счет параллельности и эффективной реализации ([FlashAttentionV2](#)). Однако, для сверхбольших последовательностей и этого уже недостаточно, и человечество разработало новый подход, выраженный в лице [модели S4](#) и [Mamba](#). Но недавно появилась статья, которая говорит о том, что в таких задачах RNN [тоже могут быть конкурентноспособны](#), однако, на задачах из этого ноутбука LRU не завелась в нормальном виде.

Подробнее про Attention вы узнаете совсем скоро!

Посимвольная языковая модель

Как обучать языковую модель на Language Modeling

Языковая модель (в нашем случае RNN) обучается на задачу предсказания следующего токена последовательности (классификации), т.е. выходы модели для i -ого токена являются предсказанием для $i + 1$ -ого токена. В качестве функционала потерь используется кросс-энтропия.



Аutoreгрессионная генерация текста при помощи обученной языковой модели

Имея на руках языковую модель, которую мы обучили предсказывать следующий токен для некоторой последовательности, мы можем с ее помощью сгенерировать продолжение затравки (prompt) по следующему алгоритму:

input : $p = (x_1, x_2, \dots, x_N)$ prompt, L max generation length

$\mathcal{X} \leftarrow p$
 $i \leftarrow N$
while ($i < L$) and ($x_i \neq \langle \text{eos} \rangle$)
 $\text{logits} \leftarrow \mathcal{M}(\mathcal{X})$
 $P \leftarrow \text{Softmax}(\text{logits})$
 $x_{i+1} \leftarrow \text{Sample}(P_i)$
 $\mathcal{X} \leftarrow \text{Append}(\mathcal{X}, x_{i+1})$
 $i \leftarrow i + 1$

return $\mathcal{X}$

Стратегии декодирования

В данном подпункте более детально разберем как, уже имея на руках посчитанные логиты модели, декодировать следующий токен последовательности. Существует большое количество способ сделать это, каждый из которых имеет свои плюсы и минусы. Сейчас разберем 3 из них: жадное декодирование (greedy decoding), лучевой поиск (beam search) и top-k сэмплирование.

Жадный алгоритм или greedy decoding

На каждом шаге декодирования выбирается наиболее вероятный токен. Наиболее простой в реализации и логичный алгоритм, который может быть уместен в некоторых сценариях, однако на практике используется редко.

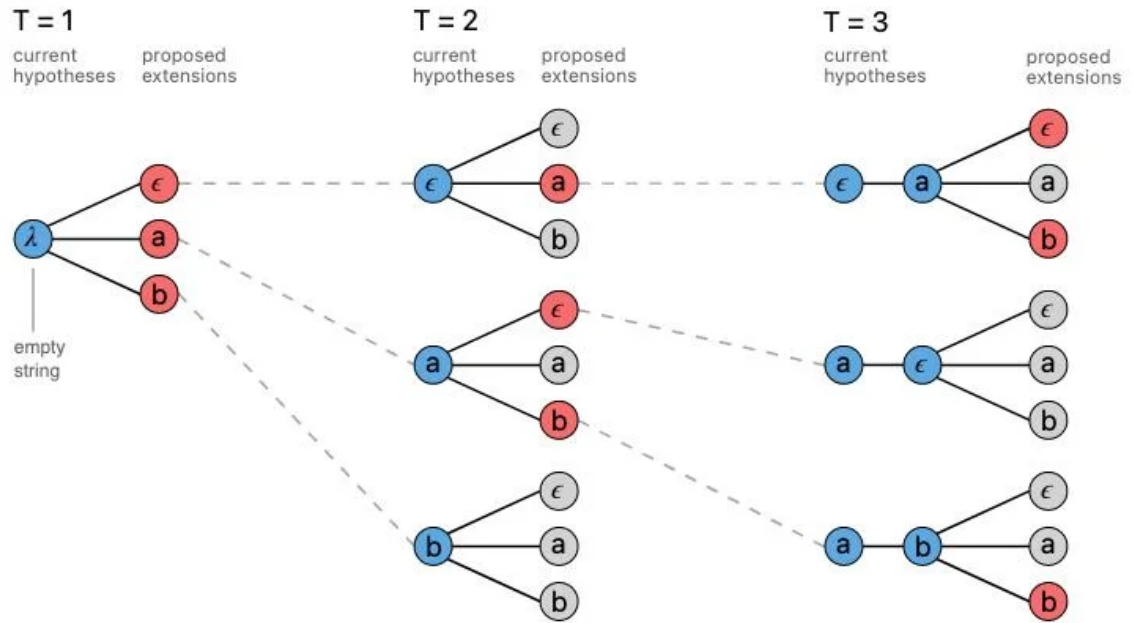
Пусть $P \in \mathbb{R}^{T \times V}$ вероятности полученные из логитов модели путем Softmax преобразования. Тогда:

$$x_{T+1} \leftarrow \text{argmax}(P_T)$$

Лучевой поиск или beam search

Источник

Главными недостатками жадного алгоритма является однообразность генерируемых текстов и риск заикливания. Алгоритм Beam search решает обе эти проблемы, при этом сохраняя высокий уровень детерминированности, что важно во многих задачах NLP.



Источник

На первом шаге генерации выбирается k наиболее вероятных токенов, выданных моделью, каждый из них добавляется к промπτу. Эти k цепочек задают начальные beams $B_0 \in \mathbb{R}^{k \times (T+1)}$:

$$B_0 \leftarrow [\text{concat}(p, x_{i_1}), \dots, \text{concat}(p, x_{i_k})].$$

На каждом следующем шаге генерации все цепочки в B подаются единым батчем в модель:

$$P_i \leftarrow \text{Softmax}(\text{model.forward}(B_i)), \quad P_i \in \mathbb{R}^{k \times (T+i) \times V}.$$

Для каждого из $k \times V$ вариантов продолжения считается логарифм совместной вероятности цепочки:

$$\log P(x_1, \dots, x_t) = \sum_{i=1}^t \log P(x_i \mid x_1, \dots, x_{i-1}).$$

Все $k \times V$ цепочек сортируются в порядке убывания совместной вероятности и оставляются только k наиболее вероятных. Эти цепочки формируют $B_{i+1} \in \mathbb{R}^{k \times (T+i+1)}$.

Генерация продолжается пока все лучи не достигнут максимальной длины L_{max} или не будут сгенерированы токены конца последовательности $\langle \text{eos} \rangle$ для всех k лучей.

Для выбора итоговой цепочки все логарифмы совместной вероятности делятся на L_j^α , коэффициент α задает величину штрафа на более короткие последовательности (обычно выбирается около 0.75). Таким образом можно регулировать, отдавать

предпочтение более длинным и развернутым или более коротким и лаконичным ответам.

$$\text{FinalScore}(j) = \frac{\log P_j(p, x_1^j, \dots, x_{L_j}^j)}{L_j^\alpha}, \quad j = 1, \dots, k$$

В итоге выбирается последовательность (beam) с максимальным FinalScore.

Top-k sampling

Следующий вариант декодирования: засэмплировать следующий токен из полученного распределения $\mathcal{P}_i$, что само по себе уже является не тривиальной задачей. Сейчас однако не будем останавливаться на вопросе как засэмплировать случайную величину из некоторого распределения и будем считать, что мы умеем это делать. Возникает другая проблема: размерность словаря зачастую в языковых моделях огромная (порядка сотен тысяч токенов), а реально подходящих из них по контексту существует лишь очень малая часть. Поэтому возникает желание ограничить число токенов, которые мы можем засэмплировать, оставив лишь k наиболее вероятных. Подобная стратегия называется top-k сэмплинг.

Сортируем $\mathcal{P}_i$ по возрастанию

$$P^* \leftarrow \text{sort}(P_i)$$

Оставляем k наибольших значений

$$P^k \leftarrow P^*[-k:]$$

Нормируем оставшиеся k значений, так чтобы числа в полученном массиве суммировались в единицу

$$P^k \leftarrow \frac{P^k}{\sum_{j=1}^k P_j^k}$$

Сэмплируем случайную величину из полученного распределения

$$x_{i+1} \leftarrow \text{Sample}(P^k)$$

Обучение языковой модели на текстах шекспира

Загрузка и препроцессинг

```
In [8]: if not os.path.exists("shake/shakespeare.txt"):
        !mkdir -p shake
```

```
!wget -q https://raw.githubusercontent.com/cedricdeboom/character-level-
```

```
In [9]: def preprocess(text):  
        return re.sub(r'^a-zA-Z0-9\s.,\'?!]', '', text)
```

```
In [10]: def save_preprocessed(path: str) -> Path:  
        """  
        Preprocess text and save it to disk.  
  
        Returns:  
            path to saved text  
        """  
        path_ = Path(path)  
        with path_.open() as file:  
            text = file.read()  
        processed = preprocess(text)  
        res = path_.parent / "processed.txt"  
        with res.open("w") as file:  
            file.write(processed)  
        return res
```

```
In [11]: path = save_preprocessed("shake/shakespeare.txt")
```

```
In [12]: def train_test_split(path: str, test_size: float = 0.1) -> tuple[Path, Path]  
        """  
        Split dataset to train and test.  
  
        Returns:  
            train_path and test_path  
        """  
        path_ = Path(path)  
        with path_.open() as file:  
            text = file.read()  
        test_num_chars = floor(len(text) * test_size)  
        test_text = text[-test_num_chars:]  
        train_text = text[:-test_num_chars]  
        train_path = path_.parent / "train.txt"  
        with train_path.open("w") as file:  
            file.write(train_text)  
        test_path = path_.parent / "test.txt"  
        with test_path.open("w") as file:  
            file.write(test_text)  
        return train_path, test_path
```

```
In [13]: train_path, test_path = train_test_split(path)
```

Токенизация

Мы воспользуемся токенизацией на уровне символов.

```
In [72]: class CharTokenizer:  
        def __init__(self) -> None:  
            self.id2token = list(string.ascii_letters + string.digits + ".,\'?!")
```

```

        self.token2id = {char: token_id for token_id, char in enumerate(self
def encode(self, txt: str) -> list[int]:
    return [self.token2id[tok] for tok in txt]

def decode(self, token_ids: list[int]) -> str:
    return [self.id2token[tok_id] for tok_id in token_ids]

```

```

In [73]: class ShakespeareTexts(Dataset):
def __init__(self, path: str, seq_length: int) -> None:
    self.seq_length = seq_length
    self.sequences = self._load_sequences(path)

    self.tokenizer = CharTokenizer()

def _load_sequences(self, path: str) -> list[str]:
    res = []
    with Path(path).open() as file:
        while True:
            sequence = file.read(self.seq_length)
            if not sequence:
                break
            res.append(sequence)
    if len(res[-1]) != self.seq_length:
        res.pop()
    return res

def __len__(self) -> int:
    return len(self.sequences)

def __getitem__(self, index: int) -> torch.LongTensor:
    """
    Returns:
        tokenized sequence
    """
    seq = self.sequences[index]
    token_ids = self.tokenizer.encode(seq)
    return torch.LongTensor(token_ids)

```

Мы будем учиться на последовательностях длины 32.

```

In [74]: train_dataset = ShakespeareTexts(train_path, seq_length=32)
test_dataset = ShakespeareTexts(test_path, seq_length=32)
len(train_dataset), len(test_dataset)

```

Out[74]: (177198, 19688)

Проверим работу токенизатора.

```

In [75]: print(train_dataset[0]),
print(train_dataset.tokenizer.decode(train_dataset[0]))

```

```

tensor([53, 58, 52, 55, 68, 68, 26, 37, 37, 44, 67, 48, 30, 37, 37, 67, 45, 3
3,
        26, 45, 67, 30, 39, 29, 44, 67, 48, 30, 37, 37, 68, 68])
['1', '6', '0', '3', '\n', '\n', 'A', 'L', 'L', 'S', ' ', 'W', 'E', 'L', 'L',
' ', 'T', 'H', 'A', 'T', ' ', 'E', 'N', 'D', 'S', ' ', 'W', 'E', 'L', 'L',
'\n', '\n']

```

Посмотрим на размер словаря.

```

In [76]: vocab_size = len(train_dataset.tokenizer.id2token)
vocab_size

```

Out[76]: 69

Построение модели

```

In [77]: device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
device

```

Out[77]: device(type='cuda', index=0)

```

In [ ]: class LanguageModel(nn.Module):
    def __init__(self, vocab_size: int, hidden_dim: int, embedding_dim: int,
                 super().__init__()

    self.hidden_dim = hidden_dim
    self.embedding = nn.Embedding(num_embeddings=vocab_size, embedding_dim=embedding_dim)
    self.rnn = nn.GRUCell(input_size=embedding_dim, hidden_size=hidden_dim)
    self.dropout = nn.Dropout(dropout_rate)
    self.lm_head = nn.Linear(hidden_dim, out_features=vocab_size, bias=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Obtain logits for next token.

        Args:
            x: tensor with token ids, shape (B,T)

        Returns:
            logits: tensor with shape (B,vocab_size, T)
        """

        B, T = x.shape
        outputs = []
        h = torch.zeros(B, self.hidden_dim, device=x.device)
        for t in range(T):
            # (B,C)
            embedded_input = self.embedding(x[:, t])
            h = self.rnn(embedded_input, h)
            outputs.append(h)

        # (B,T,C)
        outputs = self.dropout(torch.stack(outputs, dim=1))

```

```

        # (B,T,vocab_size)
        logits = self.lm_head(outputs)

        #(B,vocab_size, T)
        logits = logits.swapaxes(1,2)
        return logits

```

Утилиты для обучения

Впервые для соединения объектов в батч нам потребуется реализовать специальную функцию.

```

In [79]: def collate_sequences(batch: list[torch.LongTensor]) -> tuple[torch.LongTensor, torch.LongTensor]:
        """
        Collate function for language modeling.

        Args:
            batch: List of tensors, where each tensor is a sequence of token IDs

        Returns:
            tuple: A tuple containing:
                - inputs: Batch of input sequences.
                - targets: Batch of target sequences.
        """
        # (B,T)
        batch_tensor = torch.stack(batch)

        # (B,T-1)
        inputs = batch_tensor[:, :-1]

        # (B,T-1)
        targets = batch_tensor[:, 1:]

        return inputs, targets

```

```

In [80]: set_global_seed(SEED)
g = torch.Generator()
g.manual_seed(SEED)
test_loader = DataLoader(test_dataset, batch_size=1024, collate_fn=collate_sequences)
train_loader = DataLoader(train_dataset, batch_size=1024, collate_fn=collate_sequences)

```

```

In [81]: model = LanguageModel(vocab_size=vocab_size, hidden_dim=128, embedding_dim=EMBEDDING_DIM)
optimizer = torch.optim.AdamW(lr=3e-4, params=model.parameters())
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=500, gamma=0.5)

start_loss, start_acc = evaluate(model, device, test_loader)
print("Loss before training: ", start_loss)
print("Accuracy before training: ", start_acc)

```

```

Loss before training:  4.2601260389137074
Accuracy before training:  0.007335399981649212

```

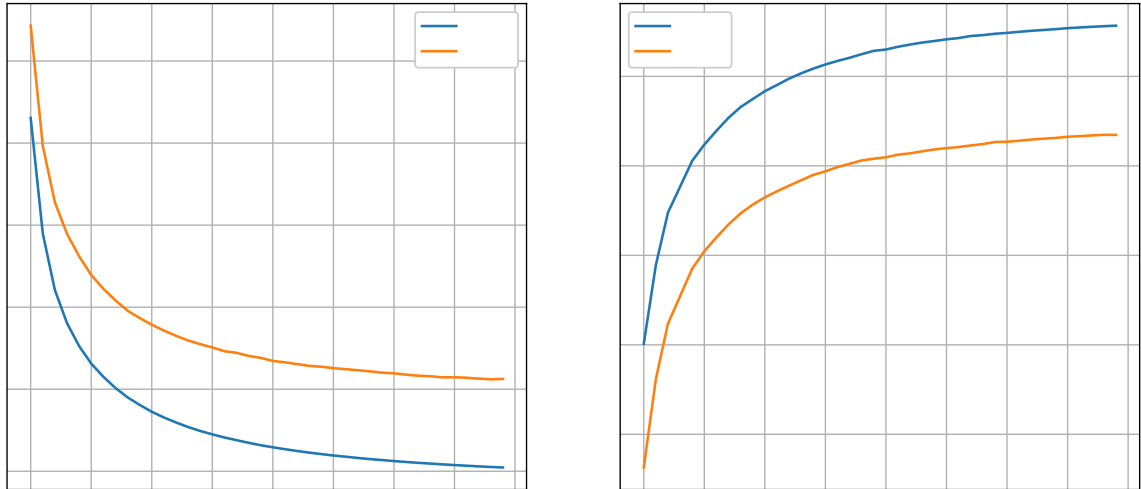
```

In [82]: train_history, test_history = train(model, device, train_loader, test_loader)

```


0% | 0/40 [00:00<?, ?it/s]

```
In [83]: plot_losses(train_history, test_history, 'Обучение языковой модели')
```



```
In [84]: @torch.no_grad()
def autoregressive_completion(
    model: LanguageModel,
    tokenizer: CharTokenizer,
    prompt: str,
    max_length: int = 100,
    device: str = "cpu"
) -> str:
    model.to(device)
    model.eval() # Set the model to evaluation mode

    # Encode the initial string
    token_ids = tokenizer.encode(prompt)
    token_ids = torch.tensor([token_ids], dtype=torch.long).to(device) # Shape: (1, ...)

    for _ in range(max_length):
        # Get logits for the next token
        logits = model(token_ids)
        next_token_logits = logits[:, :, -1]

        # Greedy decoding: select the token with the highest probability
        next_token_id = torch.argmax(next_token_logits, dim=-1) # Shape: (1,)

        # Append the predicted token to the sequence
        token_ids = torch.cat([token_ids, next_token_id.unsqueeze(0)], dim=-1)

        # Decode the generated token IDs back to a string
        generated_token_ids = token_ids.squeeze(0).tolist() # Shape: (T,)
        generated_string = tokenizer.decode(generated_token_ids)

    return "".join(generated_string)
```

```
In [85]: autoregressive_completion(model, train_dataset.tokenizer, prompt="KING OF",
```

```
Out[85]: 'KING OF SECOND SENCENIUS. I will be the see the son the son the son the so  
n the son the son the son the son'
```

Хоть модель и научилась правильно предсказывать отдельные слова, в целом качество генерации оставляет желать лучшего. Во многом это связано с выбранными стратегией декодирования и токенизатором, и с крохотным (по меркам современных LLM) числом параметров. В современных чат-ботах обычно используются более продвинутые алгоритмы декодирования, такие как top-k и top-p сэмплирование. В качестве токенизатора обычно используются разновидности BPE, который содержит в словаре не только отдельные символы, но и самые частые их комбинации.